COMPUTER NETWORKS

(CSC 258)

(B.Sc. CSIT – II/IV)

Unit-3

Part-I

Vipul Gautam

UNIT3 – DATA LINK LAYER

3.1. Function of Data Link Layer (DLL)

3.2. Overview of Logical Link Control (LLC) and Media Access Control (MAC)

3.3. Framing and Flow Control Mechanisms

Stop-and-wait ARQ, Piggybacking, Go-Back-N ARQ, Selective Repeat ARQ

3.4. Error Detection and Correction techniques

Parity checks, Cheksumming Methods, CRC, Hamming code

3.5. Channel Allocation Techniques

ALOHA, Slotted ALOHA, CSMA, CSMA/CD, CSMA/CA

3.6. Ethernet Standards → *802.3 CSMA/CD, 802.4 Token Bus, 802.5 Token Ring*

3.7. Wireless LAN → *Spread Spectrum, Bluetooth, Wi-Fi*

3.8. Overview Virtual Circuit Switching, Frame Relay & ATM

3.9. DLL Protocol → *HDLC, PPP*

FUNCTIONS OF DATA LINK LAYER

- The data link layer transforms the physical layer, a raw transmission facility, to a link responsible for node-to-node (hop-to-hop) communication.
- Specific responsibilities of the data link layer include:
  - Framing,
  - Physical addressing,
  - Synchronization,
  - Flow control,
  - Error control, and
  - Media access control.

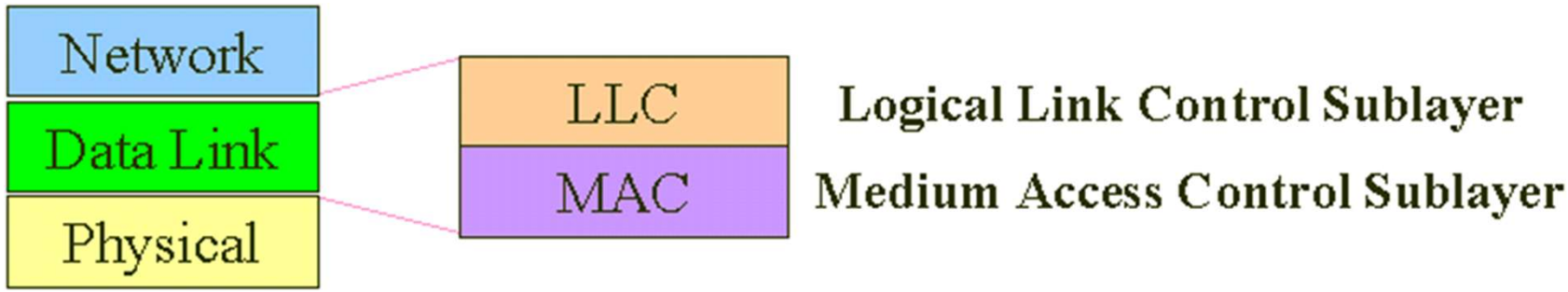
FUNCTIONS OF DATA LINK LAYER

- The data link layer divides the **stream of bits** received from the network layer into manageable data units called **frames**.
- The data link layer adds a **header** to the frame to define the **addresses** of the sender and receiver of the frame.
- If the rate at which the data are absorbed by the receiver is less than the rate at which data are produced by the sender, the data link layer imposes a **flow control** mechanism to avoid overwhelming the receiver.
- The data link layer also adds reliability to the physical layer by adding mechanisms to **detect and retransmit damaged, duplicate, or lost frames** via the help of **trailer**.
- When two or more devices are connected to the same link, data link layer protocols are necessary to determine which device has **control over the link** at any given time.

TWO SUBLAYERS

Data Link Layer has two sublayers:

- LLC (Logical Link Control)
- MAC (Media Access Control)

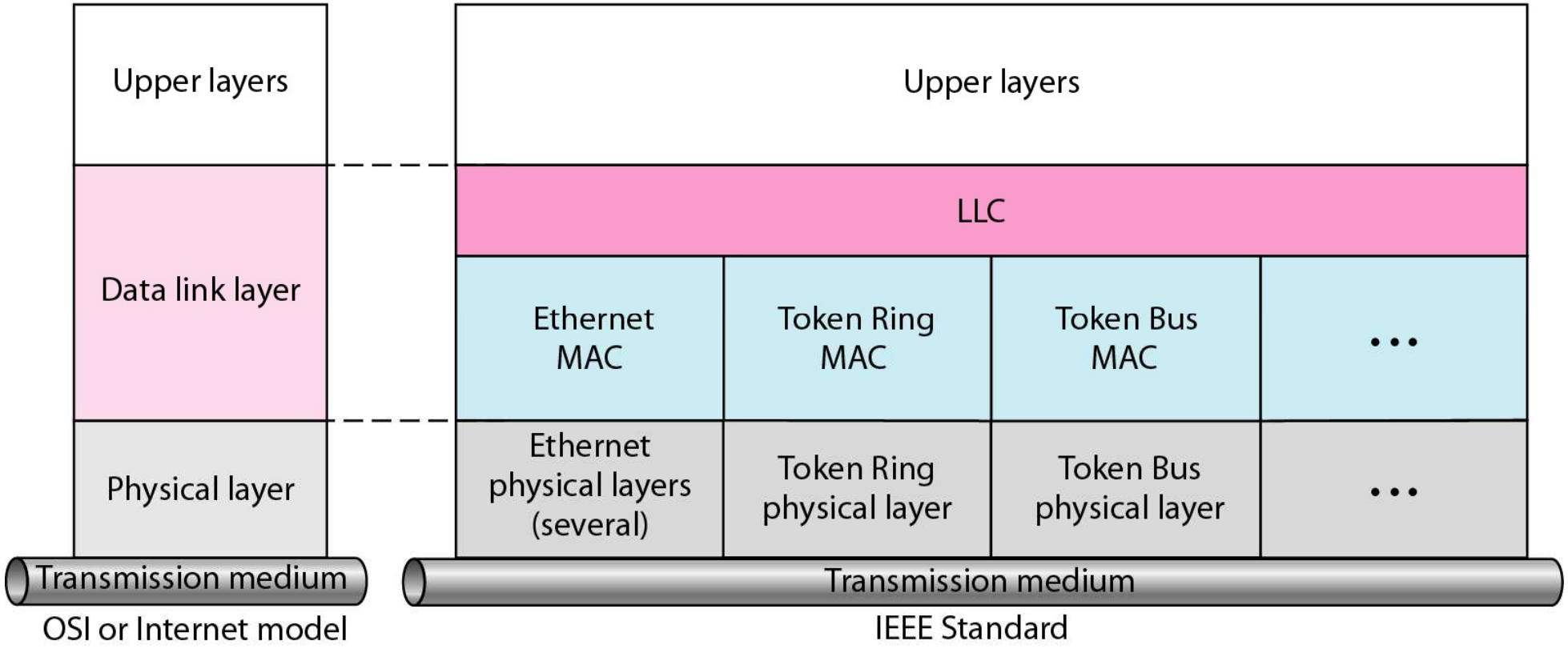


LOGICAL LINK CONTROL (LLC)

- Provides a common interface to the Network Layer.
- Handles the multiplexing.
- **Synchronization, flow control, error control,** and **part of the framing** duties are collected into logical link control sublayer.
- **Framing is handled in both** the LLC sublayer and the MAC sublayer.
- The **LLC provides one single data link control protocol for all** IEEE LANs.
- But **MAC sublayer provides different protocols for different LANs**
- A single LLC protocol can provide interconnectivity between different LANs because it makes the MAC sublayer **transparent**.

LLC & MAC PROTOCOL STACK

Single LLC protocol serving several MAC protocols



LOGICAL LINK CONTROL (LLC)

- The purpose of the **LLC is to provide flow and error control for the upper-layer protocols** that actually demand these services.
- For example, if a LAN or several LANs are used in an isolated system, LLC may be needed to provide flow and error control for the application layer protocols.
- However, most upper-layer protocols such as IP, do not use the services of LLC.
- Provides a common interface to the Network Layer.
- Handles multiplexing (SAPs).
- Provides optional connection-oriented services & flow control.
- Technology Independent.

MEDIA ACCESS CONTROL (MAC)

- Manages physical addressing (MAC addresses).
- Part of the framing function is also handled by the MAC layer (e.g. adding specific headers and trailers).
- IEEE MAC defines the specific access method for each LAN.
- For example, it defines *CSMA/CD* as the media access method for Ethernet LANs and
- Controls access to the shared physical medium (CSMA/CD for Ethernet, CSMA/CA for Wi-Fi, and the token passing method for Token Ring and Token Bus LANs.
- Handles basic error detection (CRC in the frame trailer).

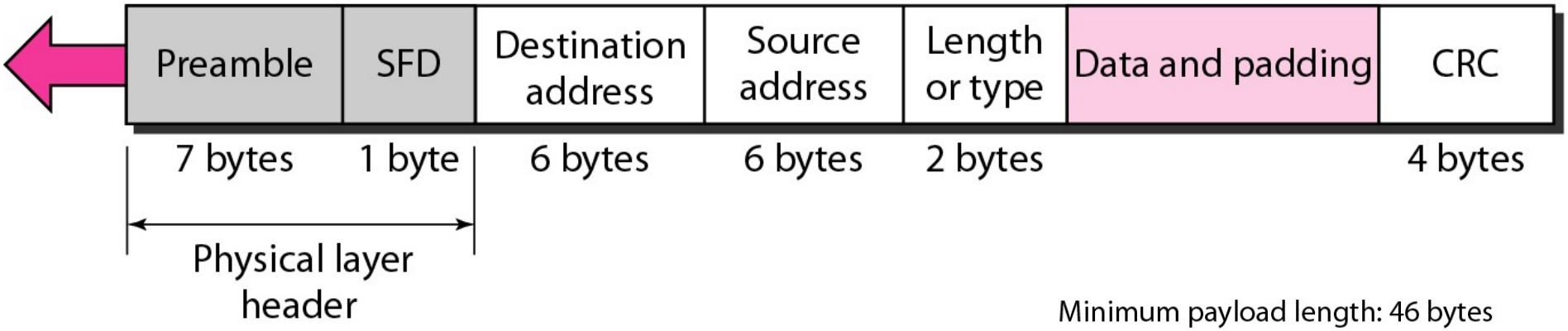
MEDIA ACCESS CONTROL (MAC)

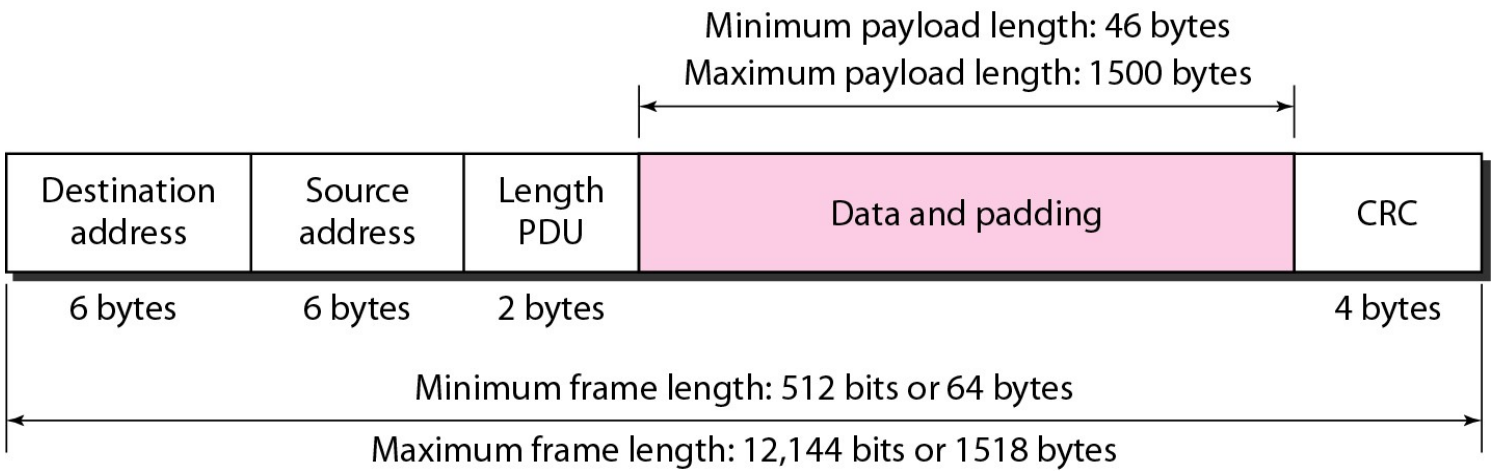
- In Standard Ethernet, the MAC sublayer governs the operation of the access method.
- It also frames data received from the upper layer and passes them to the physical layer.
- The Ethernet frame contains seven fields:
  - Preamble, Start Frame Delimiter (SFD), DA, SA, Length or Type of Protocol Data Unit (PDU), Upper-layer Data, and CRC (Cyclic Redundancy Check).
- Ethernet does not provide any mechanism for acknowledging received frames, making it what is known as an unreliable medium.
- Acknowledgments must be implemented at the higher layers.

ETHERNET FRAME

Preamble: 56 bits of alternating 1s and 0s.

SFD: Start frame delimiter, flag (10101011)





PING

Check Frames in ICMP Ping:

Packet No. 1
- Frame Length: 106 bytes which includes
ICMP Data: 64, ICMP Header: 8, IPv4 Header: 20, Ethernet: 14, No CRC

Packet No. 9
-Frame Length: 74 bytes
ICMP Data: 32, ICMP Header: 8, IPv4 Header: 20, Ethernet: 14, No CRC

Ping.pcapng

C:\WINDOWS\system32\cmd.exe

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2403:3800:323a:9ff3:8723:bc04:b312:b131
   Temporary IPv6 Address. . . . . . : 2403:3800:323a:9ff3:5927:f96:7a72:c054
   Link-local IPv6 Address . . . . . : fe80::38bd:e391:2d05:d5f1%4
   IPv4 Address. . . . . . . . . . . : 192.168.1.206
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%4
                                       192.168.1.254

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

C:\Users\vipul>ping -4 www.google.com

Pinging www.google.com [216.58.203.36] with 32 bytes of data:
Reply from 216.58.203.36: bytes=32 time=65ms TTL=57
Reply from 216.58.203.36: bytes=32 time=85ms TTL=57
Reply from 216.58.203.36: bytes=32 time=62ms TTL=57
Reply from 216.58.203.36: bytes=32 time=63ms TTL=57

Ping statistics for 216.58.203.36:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 62ms, Maximum = 85ms, Average = 68ms

C:\Users\vipul>ping -4 -l 64 www.google.com

Pinging www.google.com [142.250.182.196] with 64 bytes of data:
Reply from 142.250.182.196: bytes=64 time=61ms TTL=115
Reply from 142.250.182.196: bytes=64 time=68ms TTL=115
Reply from 142.250.182.196: bytes=64 time=59ms TTL=115
Reply from 142.250.182.196: bytes=64 time=61ms TTL=115

Ping statistics for 142.250.182.196:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 59ms, Maximum = 68ms, Average = 62ms
```

ETHERNET FRAME DETAILS

Preamble

- The first field of the 802.3 frame contains 7 bytes (56 bits) of alternating 0s and 1s that alerts the receiving system to the coming frame and enables it to synchronize its input timing.
- The pattern provides only an alert and a timing pulse.
- The 56-bit pattern allows the stations to miss some bits at the beginning of the frame.
- The preamble is actually added at the physical layer and is not (formally) part of the frame.

Start Frame Delimiter (SFD)

- The second field (l byte: 10101011) signals the beginning of the frame. The SFD warns the station or stations that this is the last chance for synchronization.
- The last 2 bits are 11 to alert the receiver that next field is the destination address.

ETHERNET FRAME DETAILS

Destination address (DA) → The DA field is 6 bytes and contains the physical address of the destination station/stations to receive the packet.

Source address (SA) → The SA field is also 6 bytes and contains the physical address of the sender of the packet.

Length or Type → The **IEEE** standard uses it as the **length** field to define the **number of bytes in the data field** while the **Ethernet** uses this field as the **type** ***(ether type)*** field to define the **upper-layer protocol** using the MAC frame.

e.g., 0x0800 for IPv4, 0x0806 for ARP, 0x86dd for IPv6.

Data → This field carries data encapsulated from the upper-layer protocols. It is a minimum of 46 and a maximum of 1500 bytes.

CRC → Cyclic Redundancy Check, the last field, aka trailer - contains error detection information, eg. CRC-32

TYPE

The Ether Type field contains two bytes of hexadecimal code. This table translates the codes to some of the more commonly used codes:

Ether Type	Protocol
0x0800	Internet Protocol, Version 4 (IPv4)
0x0806	Address Resolution Protocol (ARP)
0x8035	Reverse Address Resolution Protocol (RARP)
0x8100	VLAN-tagged frame (IEEE 802.1Q)
0x814C	Simple Network Management Protocol (SNMP)
0x86DD	Internet Protocol, Version 6 (IPv6)
0x8847	MPLS unicast
0x8848	MPLS multicast
0x8870	Jumbo Frames
0x888E	EAP over LAN (IEEE 802.1X)
0x88E5	MAC security (IEEE 802.1AE)
0x88F7	Precision Time Protocol (IEEE 1588)

ADDRESSES (UNI/MULTI/BROAD-CAST)

Unicast Address:

- A unicast destination address defines only one recipient; the relationship between the sender and the receiver is one-to-one.

Multicast Address:

- A multicast destination address defines a group of addresses; the relationship between the sender and the receivers is one-to-many.

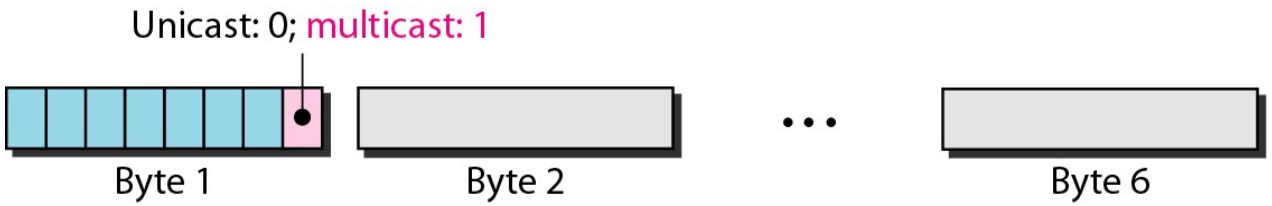
Broadcast Address:

- The broadcast address is a special case of the multicast address; the recipients are all the stations on the LAN.
- A broadcast destination address has forty-eight 1s (e.g. in case of MAC Address).

ADDRESSES (SA & DA)

SA & DA as Unicast, Multicast, and Broadcast Addresses:

- A source address is always a unicast address - the frame comes from only one station.
- The destination address, however, can be unicast, multicast, or broadcast.
- If the least significant bit of the first byte in a destination address is 0, the address is unicast; otherwise, it is multicast.



- And if there are all 1s in all bytes, the address is broadcast.

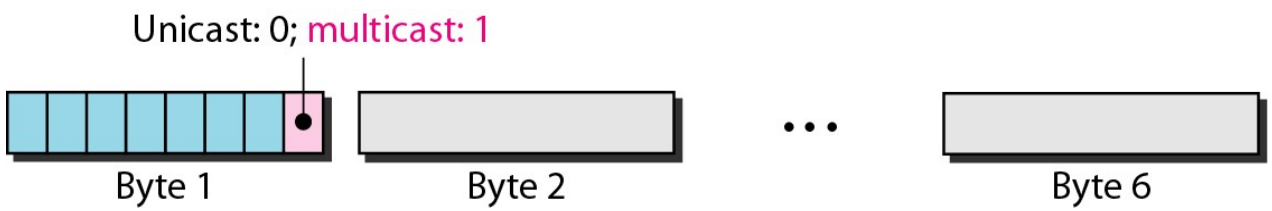
IDENTIFY THE ADDRESSES

Identify Unicast, Multicast or Broadcast addresses types from following DA's.

a. 4A:30:10:21:10:1A b. 47:20:1B:2E:08:EE

c. FF:FF:FF:FF:FF:FF d. BB:BB:BB:BB:BB:BB

- To find the type of the address:
  We need to look at the second hex digit from the left.
- If it is **even**, the address is **unicast** else (if it's ***odd***), the address is ***multicast***.
- If all digits are odd (all F's), the address is broadcast.



ANSWER

a. Unicast address because A in binary is 1010.
b. Multicast address because 7 in binary is 0111.
c. Broadcast address because all digits are F's.
d. This is also multicast address because B in binary is 1011.

UNIT3 – DATA LINK LAYER

3.1. Function of Data Link Layer (DLL)

3.2. Overview of Logical Link Control (LLC) and Media Access Control (MAC)

3.3. Framing and Flow Control Mechanisms

[Stop-and-wait ARQ, Go-Back-N ARQ, Selective Repeat ARQ] ; Piggybacking

3.4. Error Detection and Correction techniques

Parity checks, Cheksumming Methods, CRC, Hamming code

3.5. Channel Allocation Techniques

ALOHA, Slotted ALOHA, CSMA, CSMACD, CSMA/CA

3.6. Ethernet Standards → *802.3 CSMA/CD, 802.4 Token Bus, 802.5 Token Ring*

3.7. Wireless LAN → *Spread Spectrum, Bluetooth, Wi-Fi*

3.8. Overview Virtual Circuit Switching, Frame Relay & ATM

3.9. DLL Protocol → *HDLC, PPP*

DATA LINK CONTROL & MEDIA ACCESS CONTROL

- The two main functions of the data link layer are **data link control** and **media access control**.
- The function, **data link control**, deals with the ***design and procedures for communication*** between two adjacent nodes,
  i.e. node-to-node communication.
- Another function is **media access control**, or how to share the link.

- **Data link control** functions include **framing, flow control and error control**, and "***software implemented protocols***" that provide smooth and reliable transmission of frames between nodes.
- To implement data link control, we need protocols.
- Each protocol is a set of rules that need to be implemented in software and run by the two nodes involved in data exchange at the data link layer.

FRAMING

- Data transmission in the physical layer means moving bits in the form of a signal from the source to the destination.
- The **physical layer provides bit synchronization** to ensure that the sender and receiver use the same bit durations and timing.
- The data link layer, on the other hand, needs to pack bits into frames, so that each frame is distinguishable from another.
- Our postal system practices a type of framing.
  - The simple act of "inserting a letter into an envelope" separates one piece of information from another; the envelope serves as the delimiter.
  - In addition, each envelope defines the sender, and receiver addresses since the postal system is a many-to-many carrier facility.

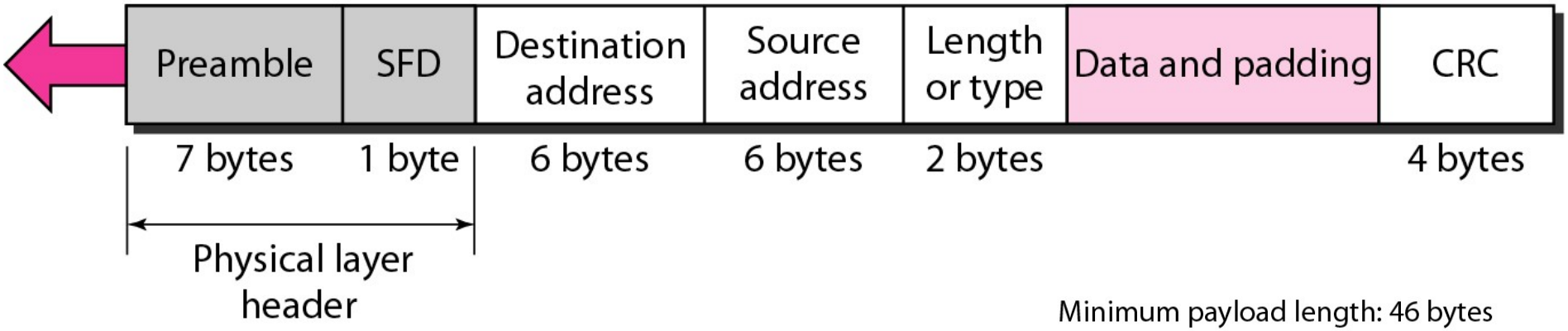
FRAMING

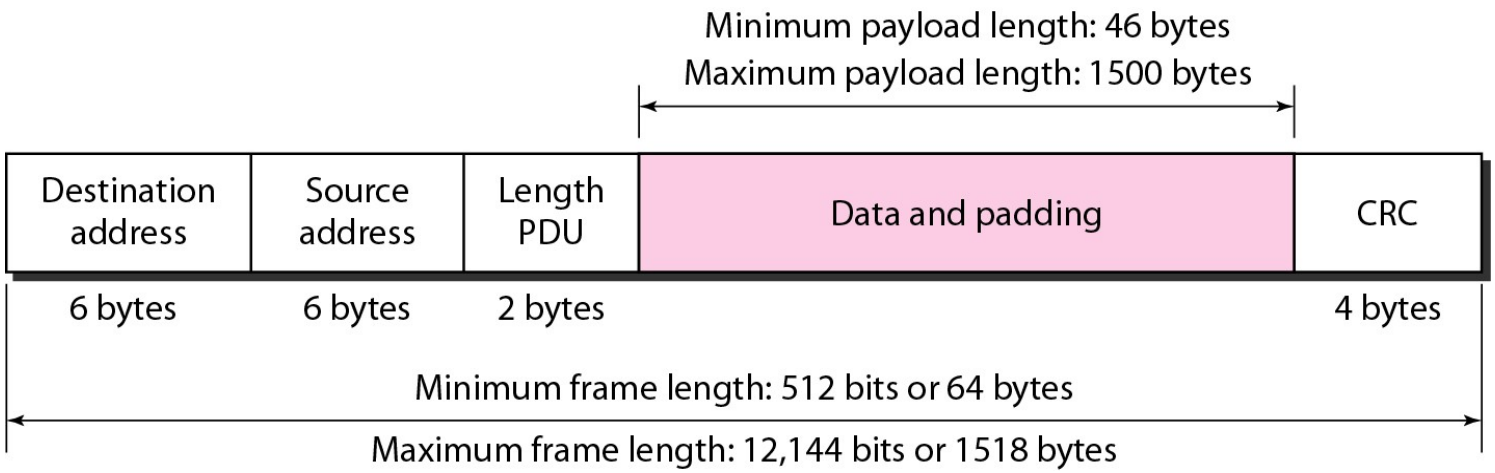
- Framing in the data link layer separates a message from one source to a destination, or from other messages to other destinations, by adding a source address and a destination address.
- The destination address defines where the packet is to go; the source address helps the recipient acknowledge the receipt.
- Although the **whole message could be packed in one frame**, that is **not normally done**.
  - One reason is that a **frame** can be **very large**, making flow and error control very inefficient, also **buffer size** if needed should be large enough.
  - Other reason is when a message is carried in one very large frame, even a **single-bit error** would require the **retransmission of the whole message**.
    - Therefore, the message is divided into smaller frames, such that a single-bit error affects only that small frame.

ETHERNET FRAME

Preamble: 56 bits of alternating 1s and 0s.

SFD: Start frame delimiter, flag (10101011)





FRAMING

Frames can be of **fixed** or **variable** size.

Fixed-Size Framing

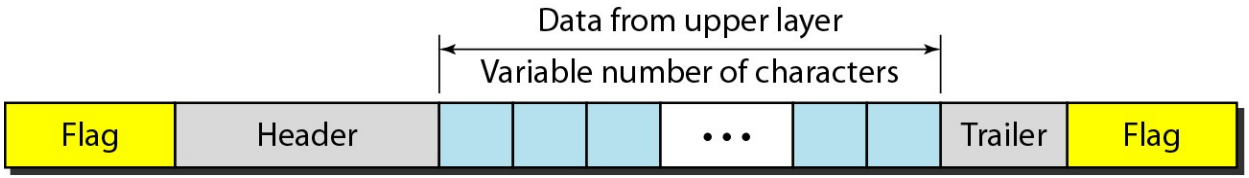
- In fixed-size framing, there is no need to define the boundaries for the frames; the size itself can be used as a delimiter.
- An example of this type of framing is the ATM wide-area network, which uses frames of fixed size called cells.

Variable-Size Framing

- Variable-size framing is prevalent in **local area networks**.
- In variable-size framing, we need a way to define the end of one frame and the beginning of the next.
- Two approaches are used for this purpose:
  - **character-oriented** approach and **bit-oriented** approach.

CHARACTER ORIENTED PROTOCOLS

- In a character-oriented protocol, data to be carried are 8-bit characters from a coding system such as **ASCII** (*Example in notepad:* ***1 char = 8 bits***).
- The **header** normally carries the **source, and destination addresses** and other **control information**.
- The **trailer** carries **error detection or error correction** redundant bits.
- All are multiples of 8 bits.
- To separate one frame from the next, an **8bit (1byte) flag** is added at the beginning and at the end of a frame.
- The flag, composed of protocol-dependent special characters, signals the start or end of a frame.



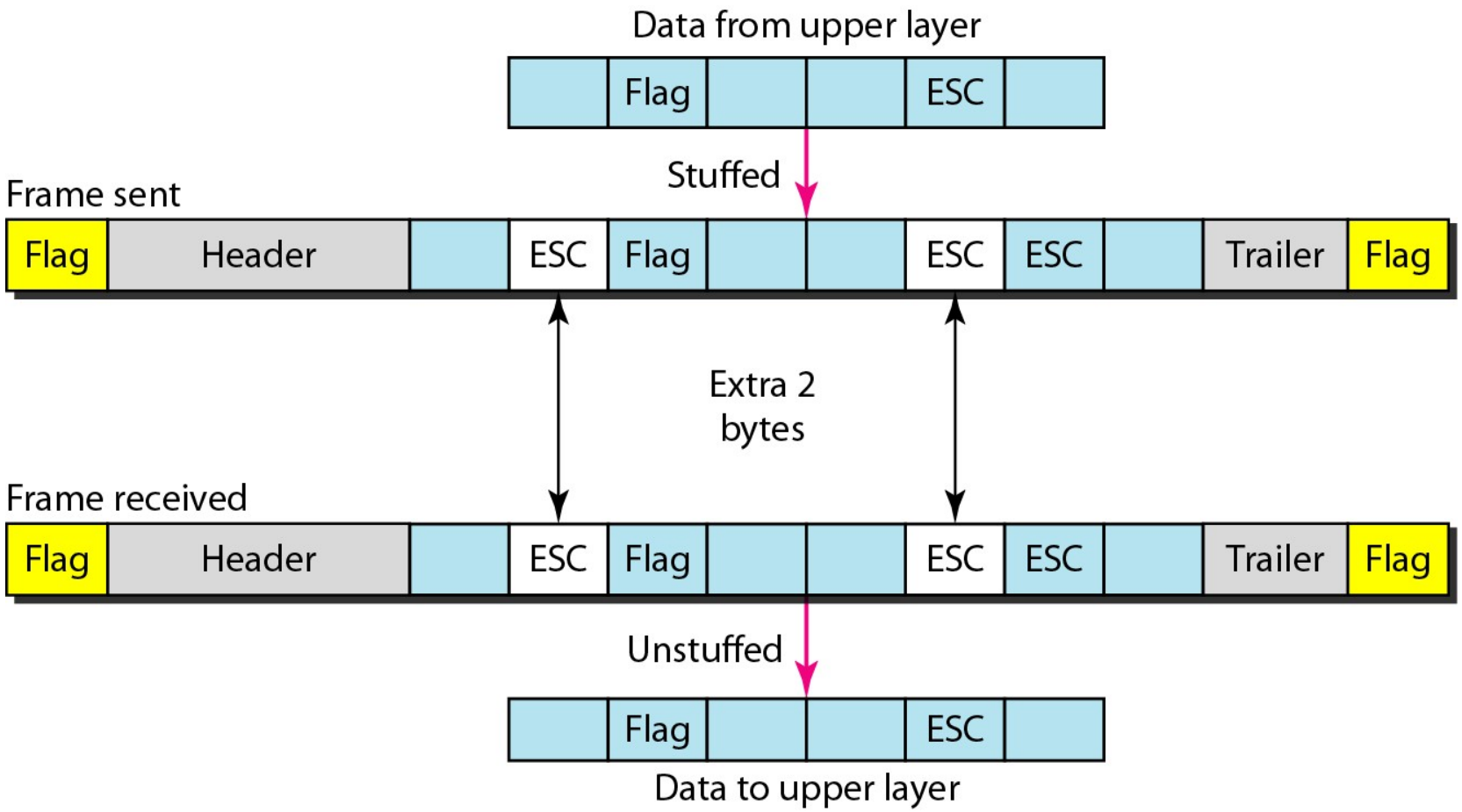
CHARACTER ORIENTED PROTOCOLS

- Character-oriented framing was popular when only the text used to be exchanged by the data link layers.
- The flag could be selected to be any character not used for text communication.
- But now we send other types of information such as graphics, audio, and video thus **any pattern used for the flag could also be part of the information.**
- If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame.
- **To fix this** problem, a **byte-stuffing strategy** was added to character-oriented framing.

CHARACTER ORIENTED PROTOCOLS

- In the byte stuffing (or character **stuffing**), a special byte is added to the data section of the frame when there is a character **with the same pattern** as the flag.
- The data section is stuffed with an extra byte. This byte is usually called the **escape character (ESC)**, which has a predefined bit pattern (***fig in next slide***).
- Whenever the receiver encounters the ESC character, character after ESC is removed from the data section and only the next character is treated as part of data.
- Byte stuffing, by the escape character allows the presence of the flag in the data section of the frame, but it creates another problem.
- What happens if the text contains **one or more escape characters followed by a flag**?

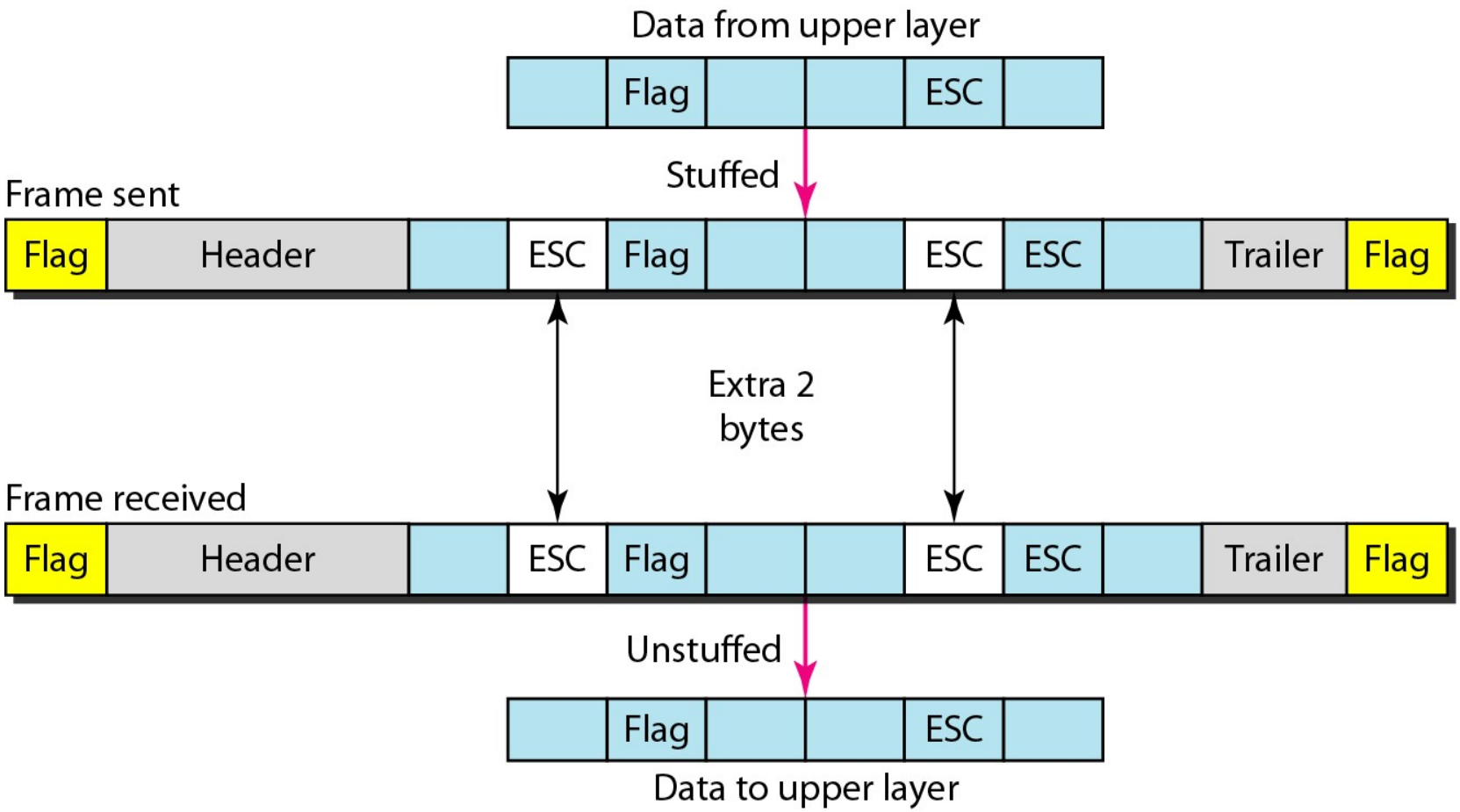
BYTE STUFFING & UNSTUFFING



CHARACTER ORIENTED PROTOCOLS

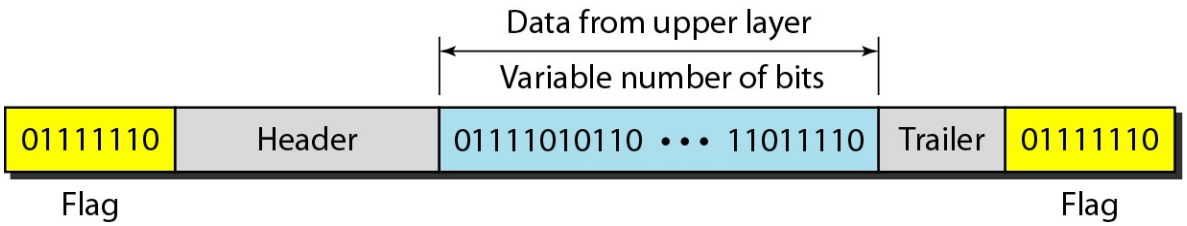
- The receiver removes the escape character, but keeps the flag, which is incorrectly interpreted as the end of the frame.
- To solve this problem, **the escape characters that are part of the text must also be marked by another escape character**.
- In other words, if the escape character is part of the text, another escape character is added to show that the second one is part of the text.

BYTE STUFFING & UNSTUFFING



BIT ORIENTED PROTOCOLS

- In a bit-oriented protocol, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on.
- However, in addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other.
- Most protocols use a special 8-bit pattern flag **01111110** as the delimiter to define the beginning and the end of the frame



BIT ORIENTED PROTOCOLS

- Here also, the flag can create the same type of problem which we got in the byte-oriented protocols.
- That is, if the flag pattern appears in the data, we need to somehow inform the receiver that this is not the end of the frame.
- We do this by stuffing 1 single bit (instead of 1 byte) to prevent the pattern from looking like a flag. The strategy is called **bit stuffing**.
- In bit stuffing, **if one 0 and five consecutive 1 bits are encountered**, an extra **0 is added**. (eg. 01111110 → 011111**0**10)
- This extra stuffed bit is eventually removed from the data by the receiver.
- Note that the ***extra bit is added after one 0 followed by five 1s regardless of the value of the next bit*** (while flag is 0 followed by six 1s).
- This guarantees that the flag field sequence does not inadvertently (accidently) appear in the frame.

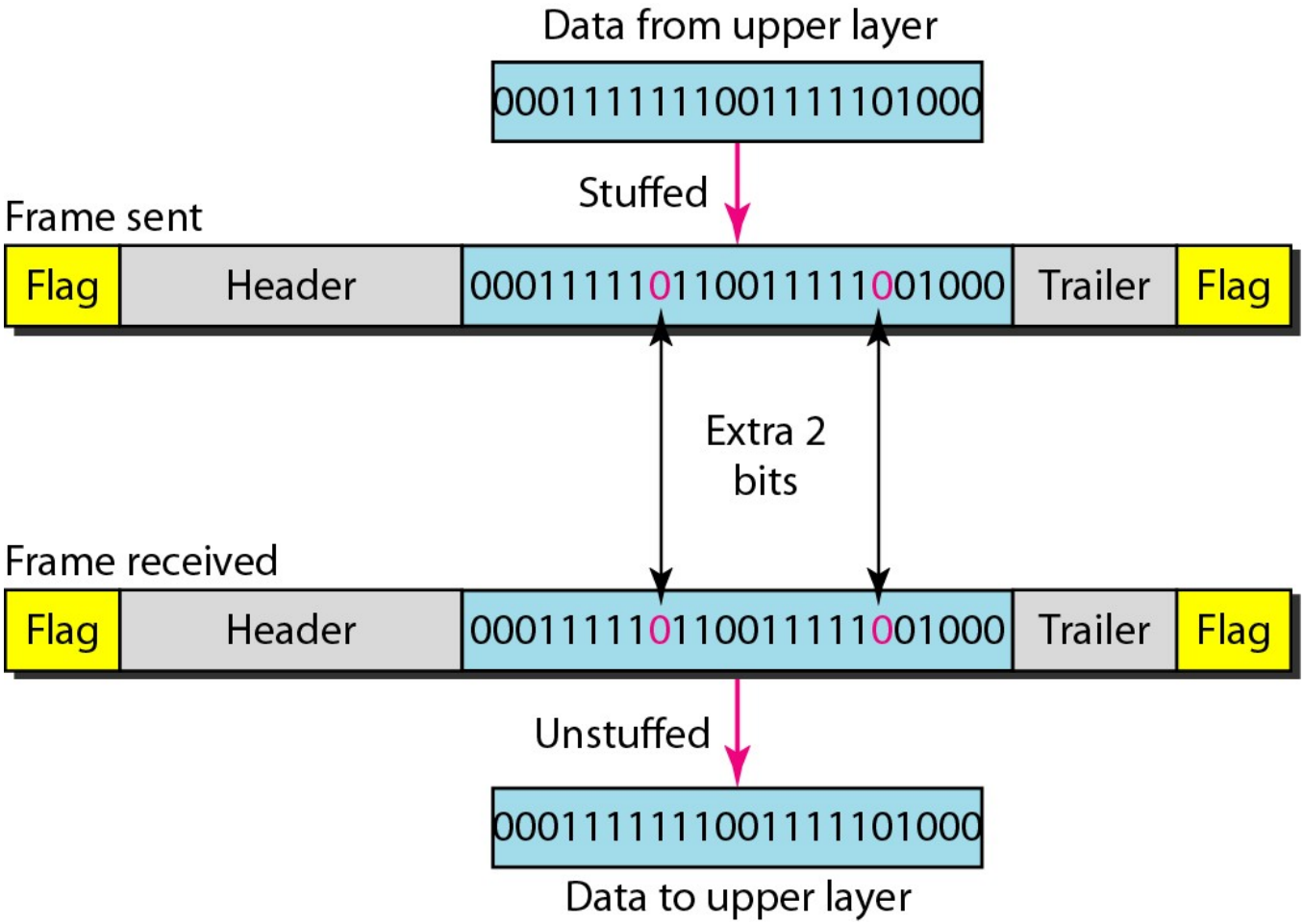
BIT STUFFING & UNSTUFFING

Note:

- Even if we have a 0 after five 1s, we still stuff a 0.
- Added 0 will be removed by the receiver.

- This means that if the flag like pattern 0**1111**10 appears in the data, it will be changed to 011111**0**10 (stuffed) and is not mistaken as a flag by the receiver.

- The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.



DATA LINK CONTROL

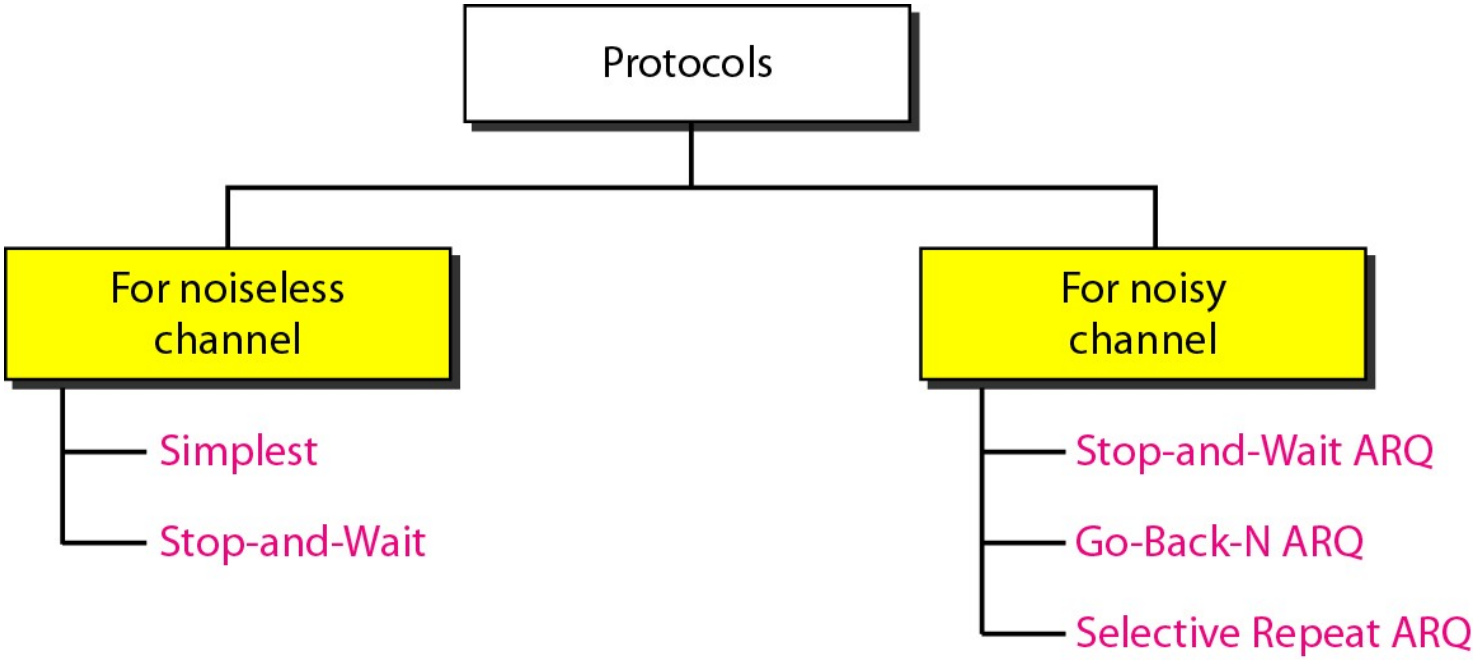
- Data communication requires at least two devices working together, one to send and the other to receive.
- Even such a basic arrangement requires a great deal of coordination for an intelligible exchange to occur.
- The most important responsibilities of the data link layer are **flow control** and **error control**.
- Collectively, these functions are known as **data link control**.

FLOW CONTROL

- Flow control is a set of procedures that tells the **sender how much data it can transmit before it must wait for an acknowledgment** from the receiver.
- The flow of data must not be allowed to overwhelm the receiver.
- Any receiving device has a **limited speed at which it can process incoming data** and a **limited** amount of **memory** in which **to store** incoming data.
- The receiving device must be able to inform the sending device before those limits are reached and to request that the transmitting device send fewer frames or stop temporarily.
- Incoming data must be checked and processed before they can be used. The rate of such processing is often slower than the rate of transmission.
- For this reason, each receiving device has a block of memory, called a ***buffer,*** **reserved for storing incoming data** until they are processed.
- **If the buffer begins to fill up, the receiver must be able to tell the sender to halt transmission until it is once again able to receive.**

FLOW CONTROL PROTOCOLS

- **Protocols** at data link layer help to combine **framing, flow control, and error control** to achieve the delivery of data from one node to another.
- The protocols are normally implemented in software by using one of the common programming languages.

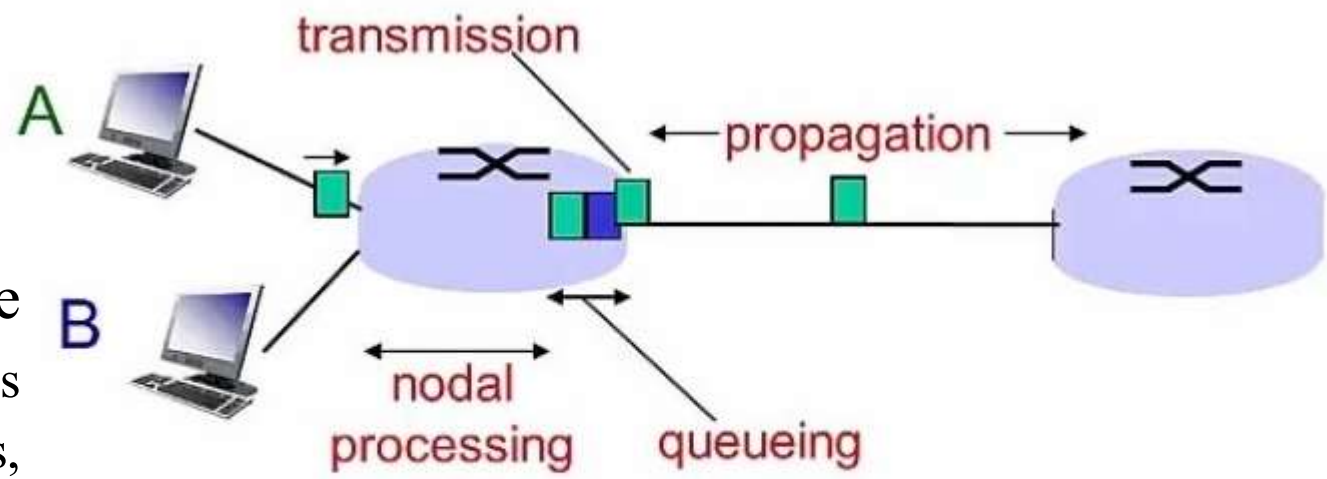


PROTOCOLS

- There is a difference between the protocols we discuss here and those used in real networks.
- All the **protocols we discuss are unidirectional** in the sense that the data frames travel from one node, called the sender, to another node, called the receiver.
- Although special frames, called **acknowledgment (ACK) and negative acknowledgment (NAK) can flow in the opposite direction** for flow and error control purposes, data flow in only one direction.
- In a real-life network, the data link protocols are implemented as bidirectional; data flow in both directions.
- In these protocols the flow and error control information such as ACKs and NAKs are included in the data frames in a technique called **piggybacking**.

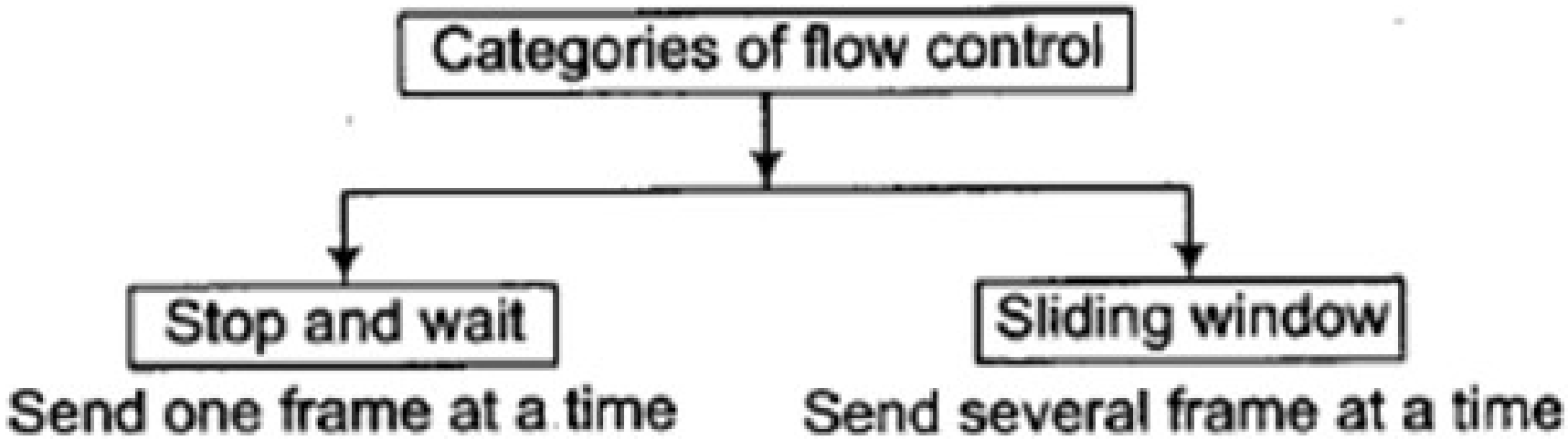
DELAYS

- **Nodal Processing Delay:**
  Receiver processes the frame (examining a packet's header, checking for errors, determining the next hop).
- **Queuing Delay:**
  Packet is queued before transmitting.
- **Transmission Delay:**
  Time taken by Sender to make frame ready to send (i.e., to keep in transmission media).



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

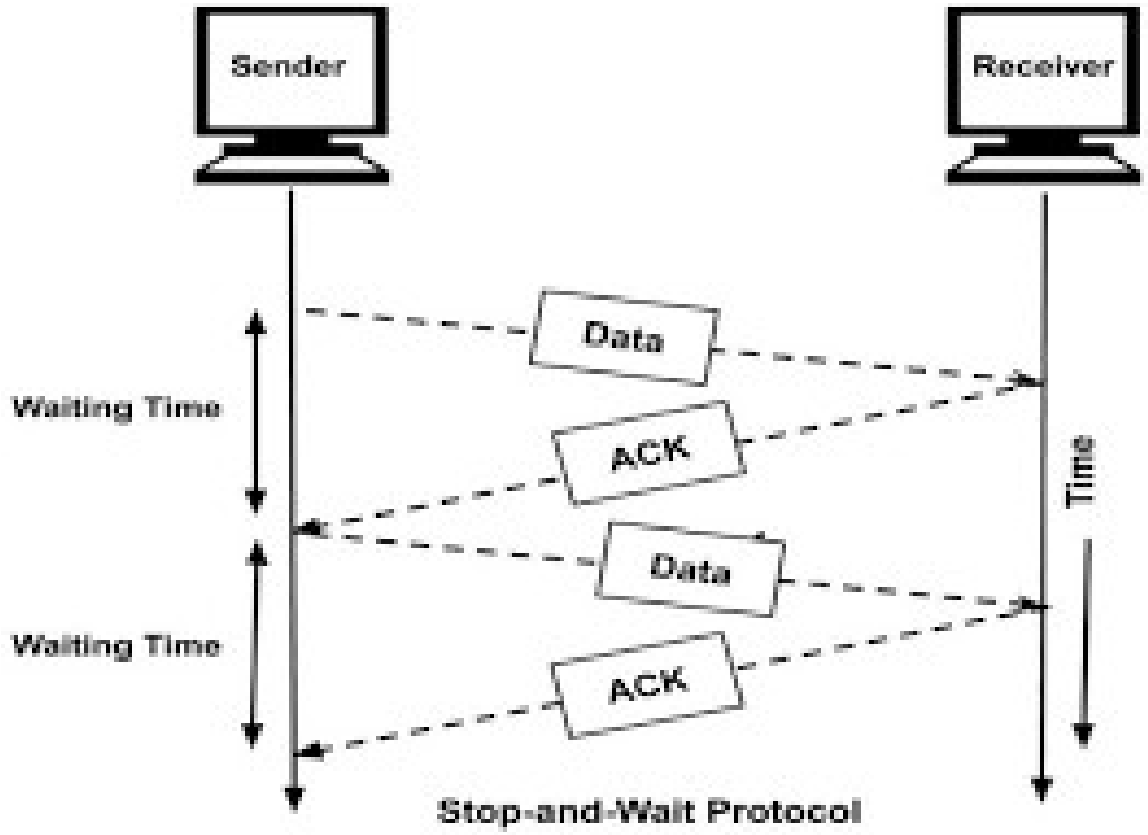
- **Propagation Delay:**
  Time taken for frame to reach from one node to the other (i.e. frame travelled time).
- **Nodal Delay (d_{nodal}):**
  Total time taken from a node receiving the frame to handing over it to another node .


Categories of flow control
Stop and wait
Send one frame at a time
Sliding window
Send several frame at a time

STOP AND WAIT FLOW CONTROL

- It’s the simplest form of flow control.
- The source transmits a data frame.
- After receiving the frame, the destination indicated its willingness to accept another frame by sending back an ACK frame, which acknowledges the just received frame.
- **The source must wait until it receives the ACK frame before sending the next data frame.**
- This time is called as WAITING TIME.

STOP AND WAIT FLOW CONTROL



SLIDING WINDOW FLOW CONTROL

- With the use of multiple frames for a single message, the stop and wait protocol does not perform well.
  - i.e., only 1 frame at a time can be in transit

- To make it more efficient, we need to allow multiple frame to be in transit at the same time, **here comes the Sliding Window Flow control**.

SLIDING WINDOW (SENDER)

- To keep track of the frames, sender station sends sequentially numbered frames
- Since, the sequence number to be used occupies a field in the frame, it should be of limited size.
- **If the header of the frame allows k bits**, the sequence numbers range from 0 to "$2^K - 1$"
- **Sender maintains a list of sequence numbers that it is allowed to send which is termed as SENDER WINDOW.**
- **The size of the sender window is at most $2^K - 1$.**
- The sender window is provided with a buffer equal to the window size.

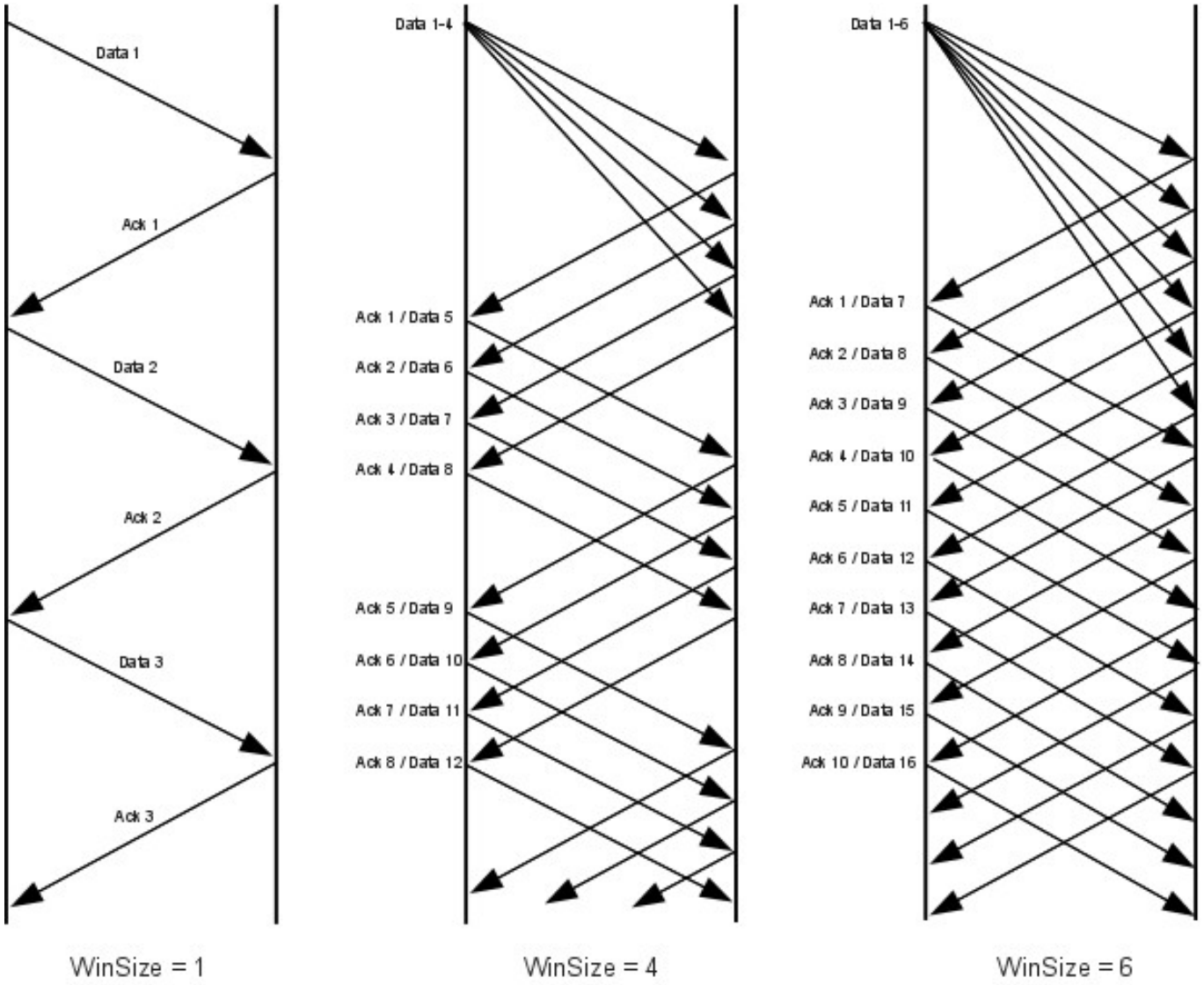
SLIDING WINDOW (RECEIVER)

- Receiver also maintains window size of 1.
- The **receiver acknowledges a frame by sending an ACK frame** that includes the sequence number of the next frame expected.
- This also explicitly announces that it is prepared to receive the next N frames, beginning with the number specified.
- This scheme can be used to acknowledge multiple frames.

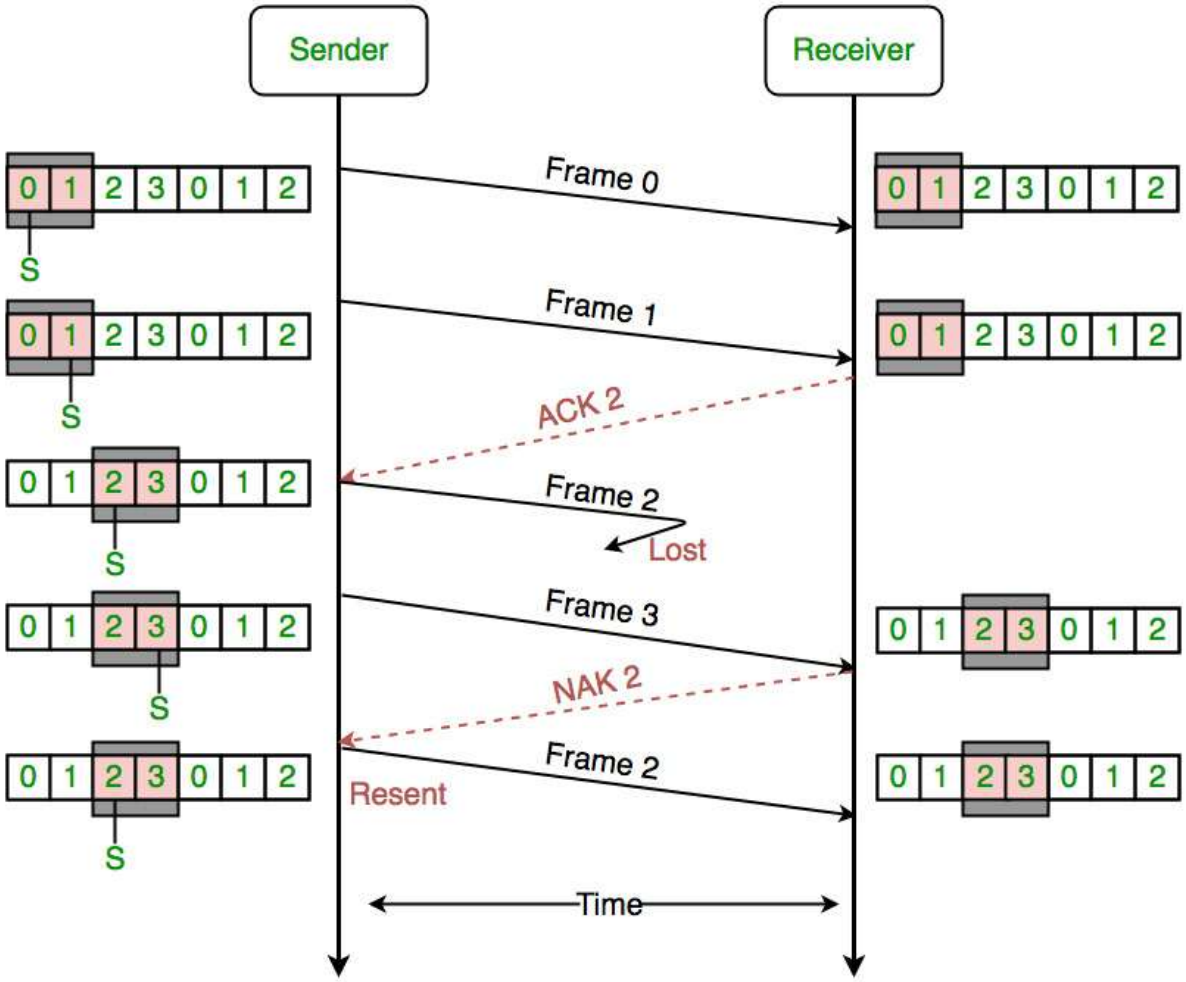
  ie. **it could receive frames 2, 3 and 4 but withhold ACK until frame 4 has arrived**.

  By returning an ACK with sequence number 5, it acknowledges the frames 2, 3 and 4 at one time.
- As window size is 1, the receiver needs a buffer of size 1 only as it receives 1 frame at a time.

SLIDING WINDOW



SLIDING WINDOW



PIGGYBACKING

- The actual window size need not be maximum possible size for a given sequence number length.
  - ie. For a 3 bit sequence number, a windows size of 4 can be configured
- If two stations exchange data, each need to maintain two windows.
- **To save communication capacity, a technique called PIGGYBACKING is used**.
- In **Piggybacking**, **each frame includes a field that holds the sequence number of that frame along with a field that holds the sequence number used for the acknowledgement, ACK**.
- But, if a station has an ACK but no data to send, it sends a separate ACK frame.

BACKWARD ERROR CONTROL

Model of frame transmission:

- Data are sent as a sequence of frames.
- Frames arrive at the same order as they are sent.
- Each transmitted frame suffers an arbitrary and variable amount of delay before reception.

In addition to above, following two types of error may occur:

- **Lost Frame:**
  - A frame fails to arrive at the other side.
  - A noise burst may damage a frame to such an extent that it is not recognizable at the receiving end.
- **Damaged Frame:**
  - A recognizable frame does arrive, but with error in some of the bits.

BACKWARD ERROR CONTROL

Most common techniques for error control are based on any or combination of the following:

- **Error Detection**
- **Positive Acknowledgement**

  The destination returns a positive acknowledgement for successfully received error-free frames.
- **Retransmission after timeout**

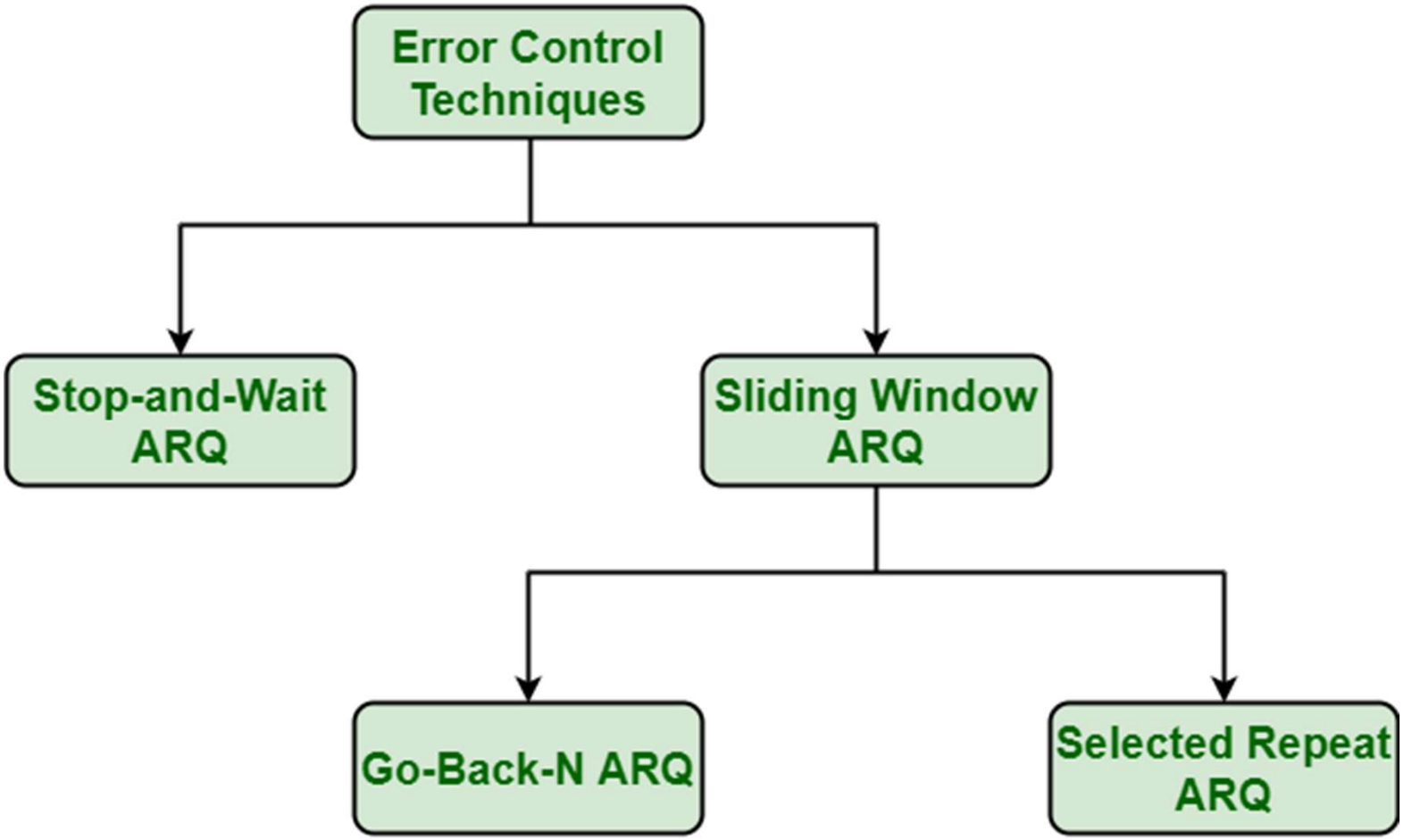
  The source retransmits a frame that has not been acknowledged after a predetermined amount of time.
- **Negative acknowledgement and retransmission**

  The destination returns a negative acknowledgement to frames in which an error is detected. The source retransmits such frames.

ERROR CONTROL

- **Error control is both error detection and error correction**.
- It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender.
- **In** the **data link layer**, the term ***error control*** **refers** primarily to methods of **error detection and retransmission**.
- Error control in the data link layer is often implemented simply:
  - **Any time an error is detected in an exchange, specified frames are retransmitted.**
  - This process is called **Automatic Repeat Request (ARQ).**

ERROR CONTROL TECHNIQUES



STOP AND WAIT ARQ

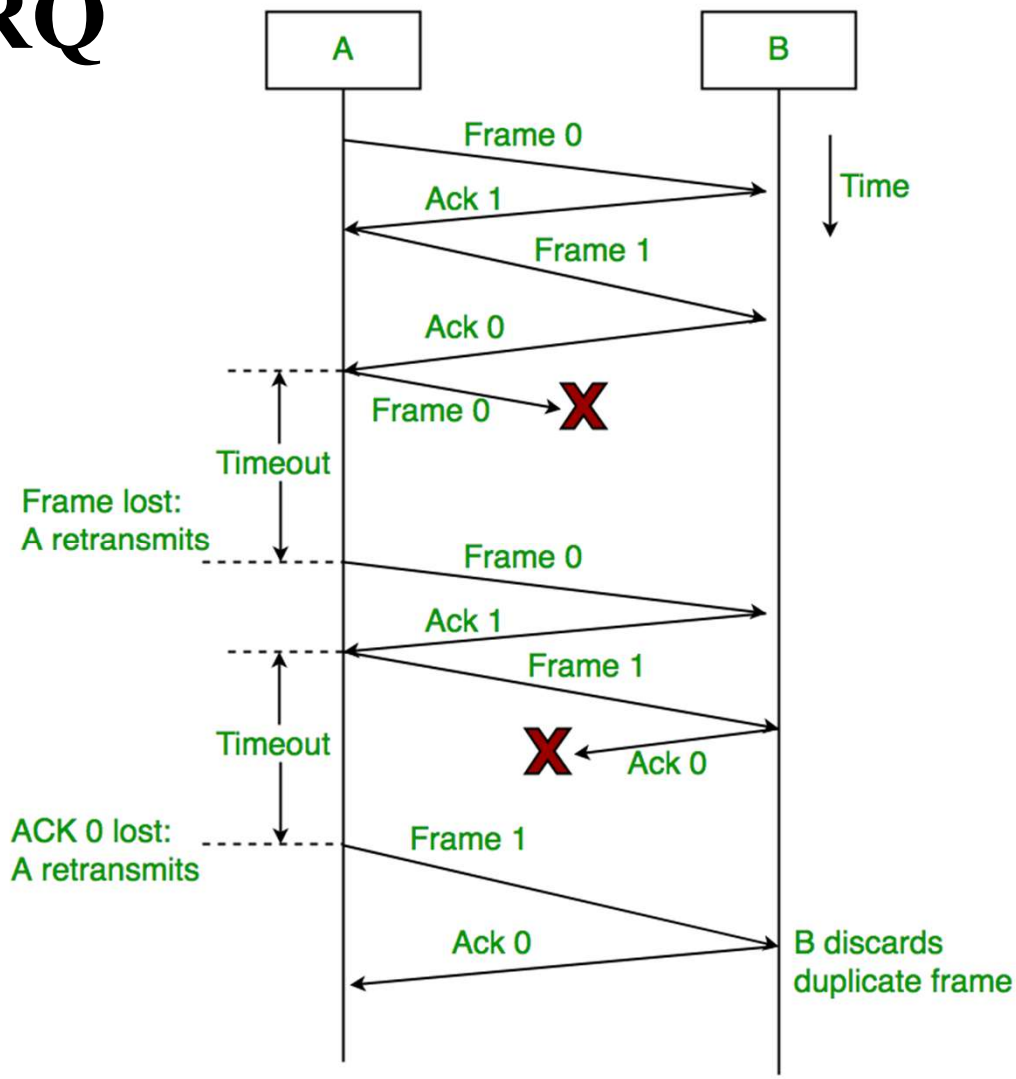
- It's based on the stop and wait flow control technique.
- The source stations transmits a single frame and then waits for an acknowledgement (ACK).
- **No other data frame can be sent until the destination station's reply arrives at the source station.**
- **To take care of lost and damaged frames**, the stations are equipped with:
  - **A Timer:** If no recognizable ACK is received **when the timer expires** at the end of the time-out interval, **then the same frame is sent again.**
  - It requires that the **transmitter maintain a copy of a transmitted frame** until an ACK is receive for it.

STOP AND WAIT ARQ

- The ACK frame may be damaged.
- The sender will timeout and resend the same frame.
- Since the receiver had already received it, it receives the same frame twice.

- Identifying the duplicate frames:
  - A numbering scheme called Modulo-2 is used.
  - Here, each frame is alternately labeled with 0 or 1, and positive acknowledgement are of the form ACK0 and ACK1.

STOP AND WAIT ARQ



STOP AND WAIT ARQ

- Advantages:
  - The main advantage of stop and wait ARQ is its simplicity.
  - It requires minimum buffer size

- Disadvantages:
  - It makes highly inefficient use of communication links, particularly when Propagation Delay is too large.

GO BACK N ARQ

- It’s based on sliding window protocol.
- It is more commonly used.
- Here, a station may send a series of frames sequentially up to a maximum number
- The number of unacknowledged frames outstanding is determined by window size, using the sliding-window flow control technique.
- In the case of no error, the destination will acknowledge incoming frames as usual

GO BACK N ARQ

- If the destination detects error in a frame, or it receives a frame out of order, it sends a NAK for that frame (using reject or REJ frame).
- The destination will discard the frame in error and all future frames until the frame in error is correctly received.
- **The source station, on receiving the REJ via NAK, must retransmit the frame in error plus all succeeding frames.**

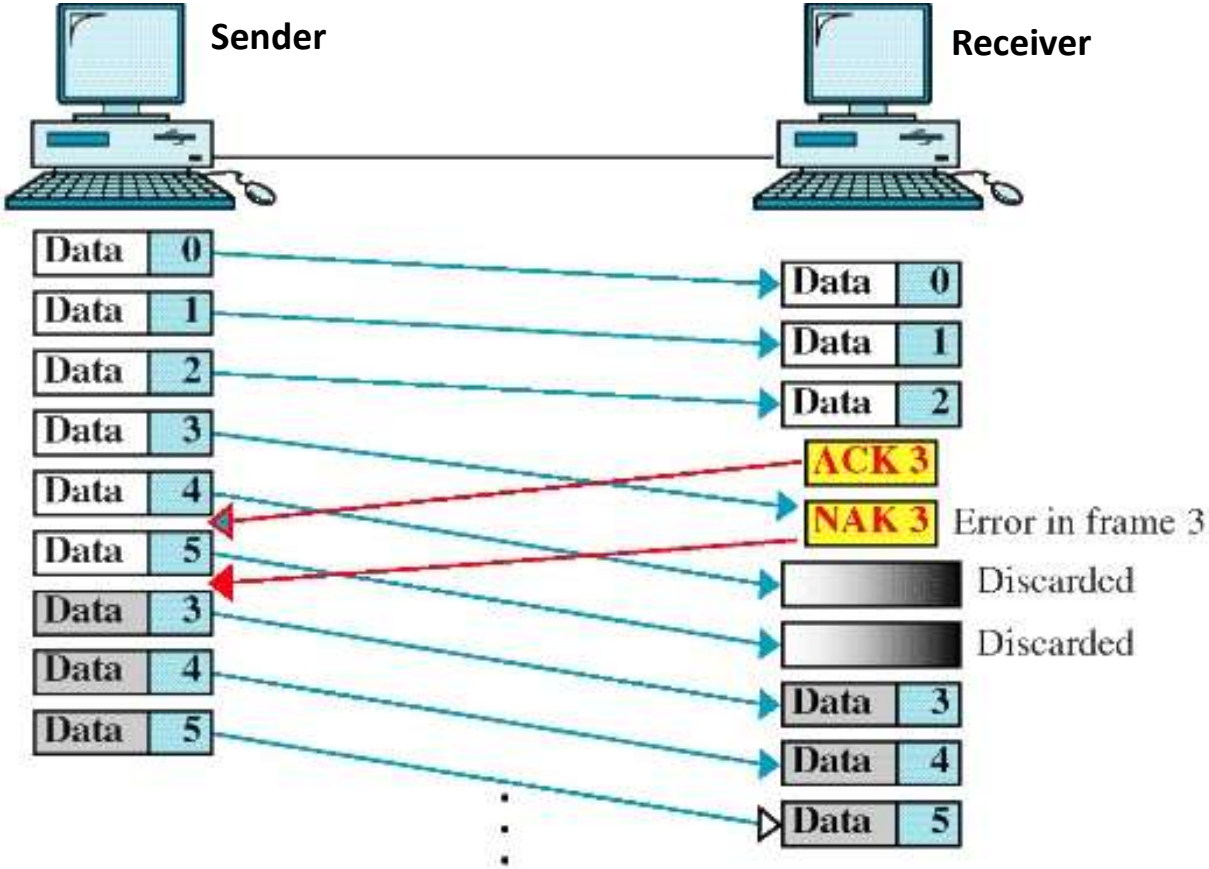
GO BACK N ARQ

In the figure,

-Receiver gets ACK3, which means it has received frame upto no. 2 and is waiting for no. 3

-It then receives frame no. 3, as it's with error so it replies back NAK to sender for frame no. 3

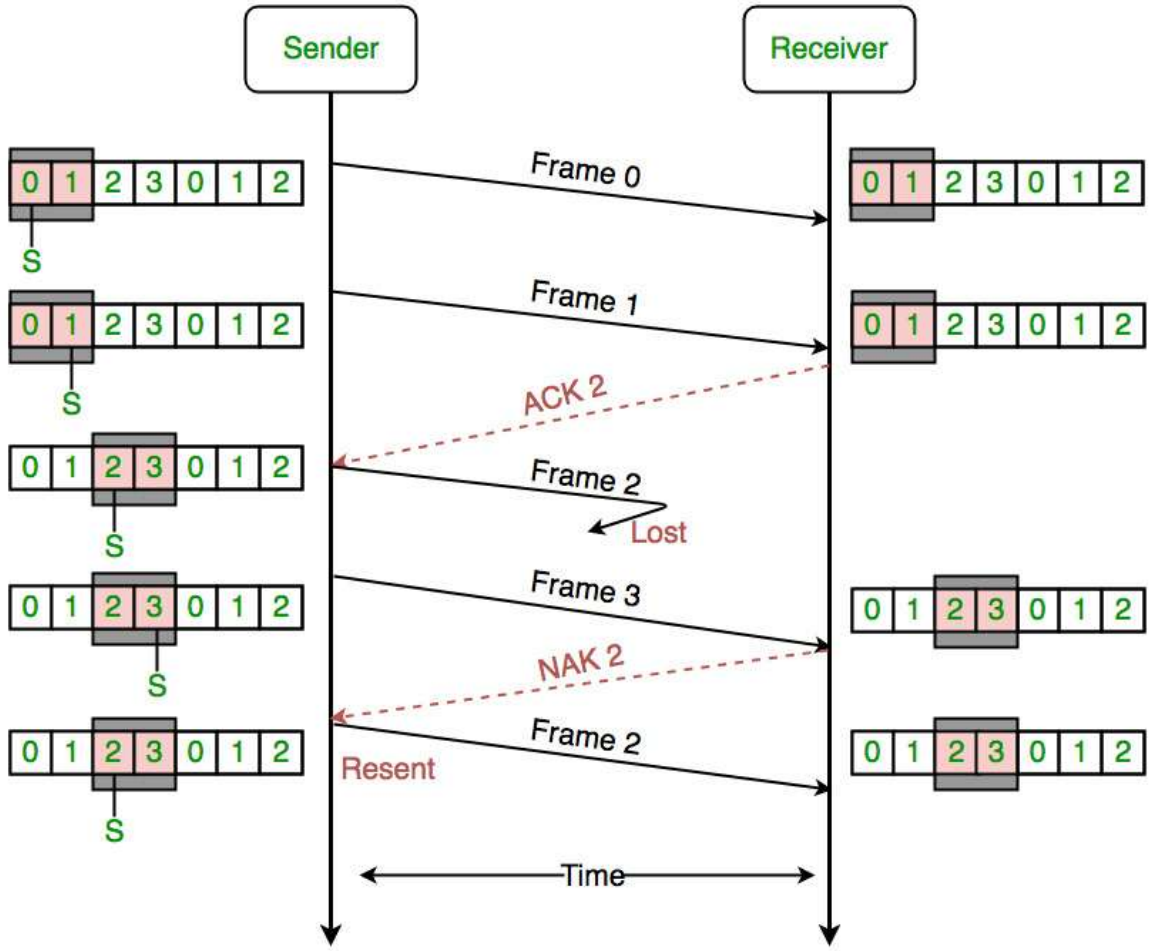
-Sender then goes back to frame no. 3 (Go Back N → Go Back 3, as N=3) , and retransmits it again.



SELECTIVE REPEAT ARQ

- In this case, only those frames are retransmitted for which the negative acknowledgement has been received.
  i.e. either SREJ (Selective REJect) or timeout might have occurred
- It's more efficient than Go Back N ARQ
- **Receiver requires storage buffers to contain "out of order frames" until the frame in error is correctly received.**
- **Receiver** must also have the **appropriate logic circuitry** needed for **reinserting the frames in the correct order.**
- **Transmitter** is also more complex because it **must be capable of sending frames out of sequence**.

SELECTIVE REPEAT ARQ



ERROR DETECTION AND CORRECTION

- Due to signal **Attenuation**, **Distortion**, **Noise** and **Interference**, errors during transmission might be inevitable, leading to corrupted transmitted bits.
- Longer the frame size, higher is the probability of single bit error, thus lower is the probability of receiving the frame without error.
- This clearly emphasizes the need for error detection and error correction.

ERROR TYPES

- **Single Bit Error**:
  - Here, only one bit gets corrupted.
  - It's common in parallel transmission.

- **Multiple Bit Error:**
  - Here, many single bit errors get occurred.

- **Burst Error** :
  - Here, more than one bits get corrupted.
  - It's common in serial transmission of data.
  - It occurs when the duration of noise is longer than the duration of one bit.

ERROR DETECTION TECHNIQUES

Redundancy

Here, additional bits are added at the end of each data unit to facilitate detection and correction of errors.

Techniques

- Simple parity check
- Two-dimensional parity check
- Checksum
- CRC (Cyclic Redundancy Check)

SIMPLE PARITY CHECK

- It's the simplest and the most popular error detection scheme.
- The method is to append a bit to the end of the data which is called as parity bit.
- These are of two types:
  - **Even Parity (No. of 1 as even)**
    - Appending either 0 or 1 at the end of data to make total number of 1 bits even, including the added parity bit.

      eg. 10010 → 10010**0** **(Sender)** 10011 → 10011**1** **(Receiver)**
  - **Odd Parity (No. of 1 as odd)**
    - Appending either 0 or 1 at the end of the data to make total number of 1 bits odd, including the added parity bit.

      eg. 10010 → 10010**1** 10011 → 10011**0**

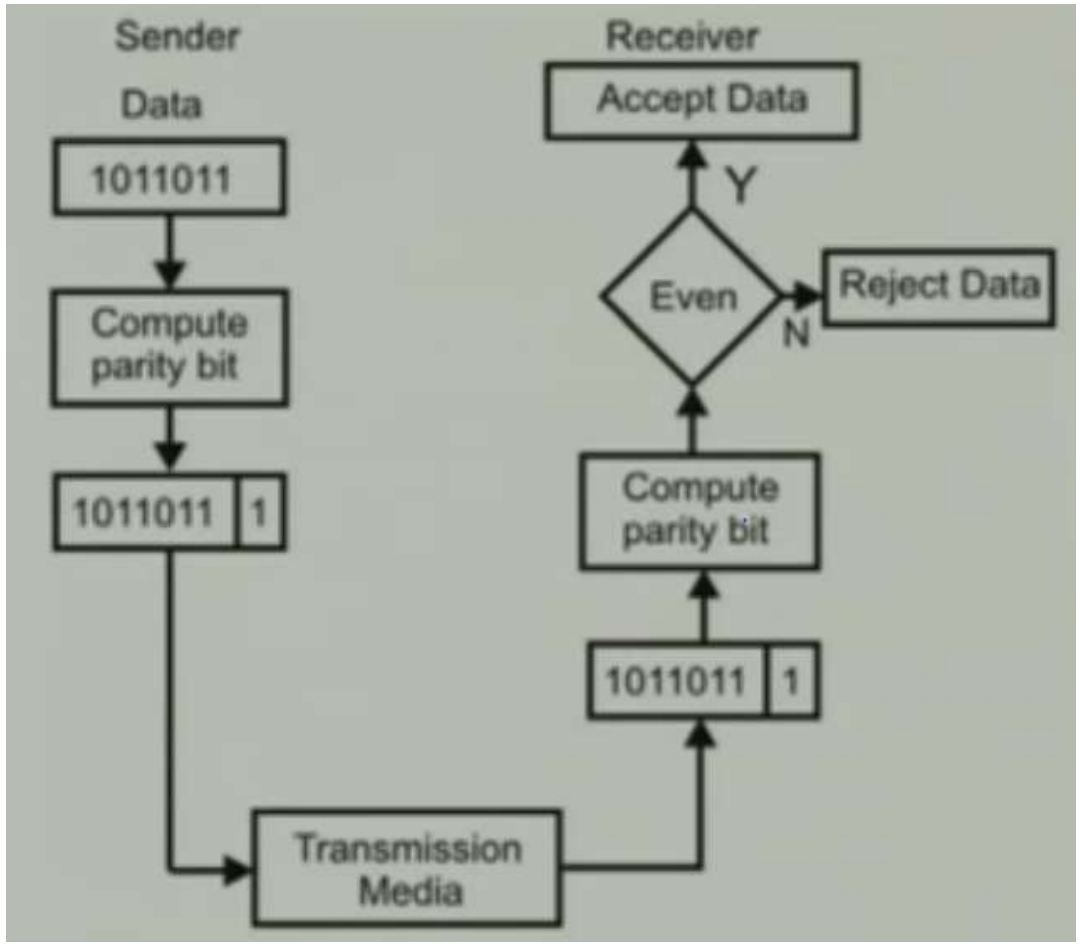
SIMPLE PARITY CHECK

General Usages:

- Even Parity is used in Synchronous Communication
  - Real time Communication
- Odd Parity is used in Asynchronous Communication
  - Sporadically (occasionally; at irregular intervals) Communication

- Simple parity check can detect all single bit errors.
- It can also **detect burst errors**; only if the number of bits in the error is odd **but can't detect if there are even number of bits inversion**.

SIMPLE PARITY CHECK



TWO DIMENSIONAL PARITY CHECK

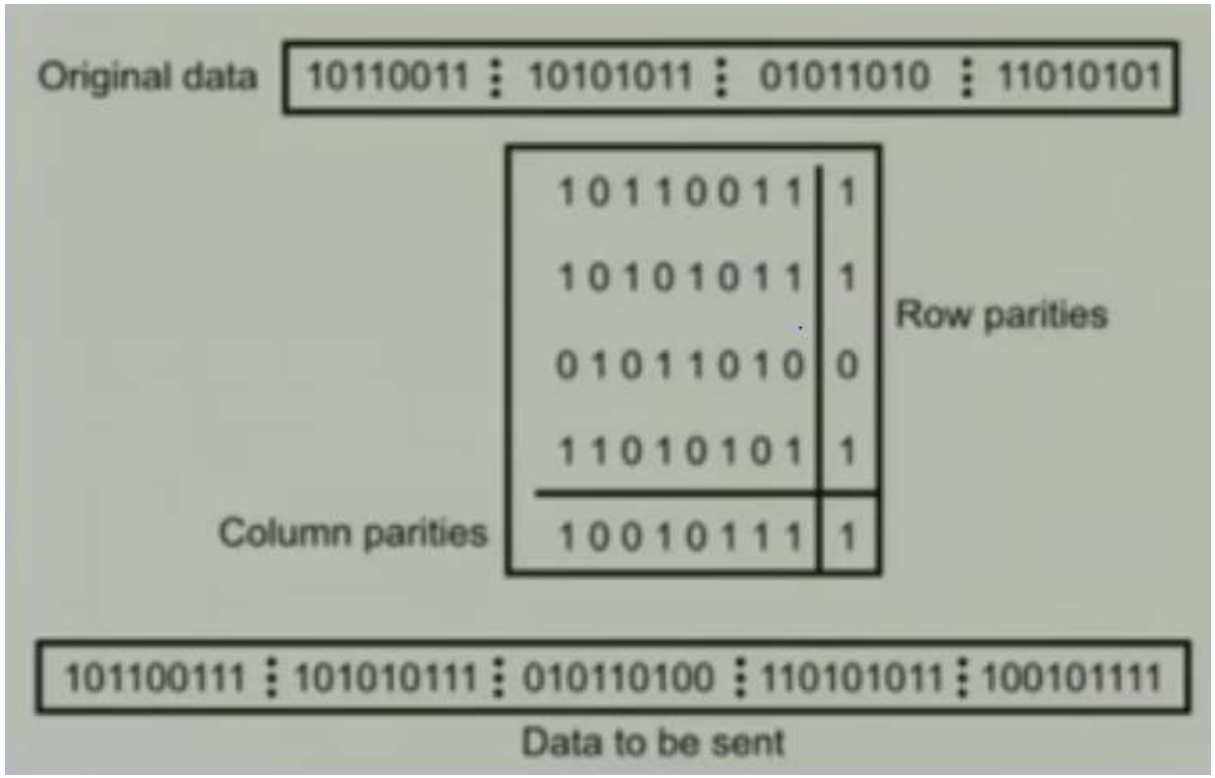
- Performance of simple parity can be improved by using two dimensional parity check, which organizes the block of bits in the form of a table.
- Parity check bits are calculated for each row, which is equivalent to a simple parity check bit.
- Parity check bits are also calculated for all columns.
- Both are sent along with the data.
- At the receiving end, these are compared with the parity bits calculated on the received bits.

TWO DIMENSIONAL PARITY CHECK

- Here, data to be sent are divided into 4 blocks or segments.
- It's the case of Even Parity.
- Parity bits are calculated for each row and column.

Additional bits to sent:

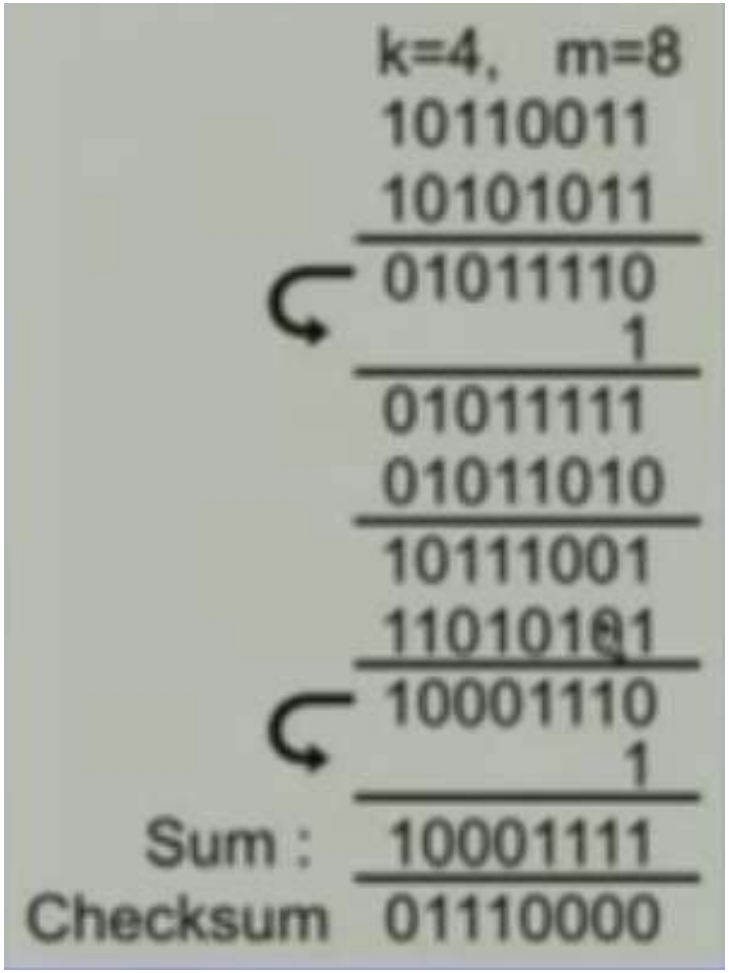
- 1 bit for each block
- 1 extra block with same number of bits.
- For 32 bit data, 13 bits extra for parity check (1 for each block; 9 for parity)



CHECKSUM

Sender's End:

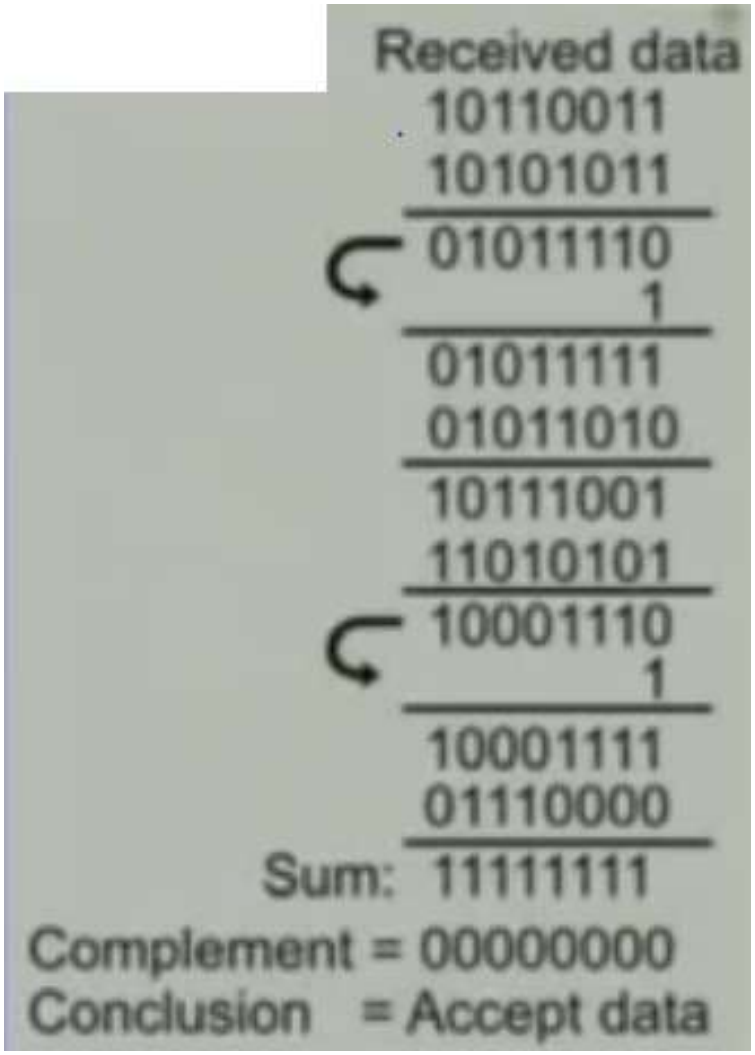
- The data is divided into **k** segments (data blocks) each of **m** bits.
- The segments are added.
- Carry observed is further added to the sum.
- **Checksum** is calculated by taking 1's complement (all 1s flipped to 0s and all 0s flipped to 1s) of the total sum.
- The checksum segment is also sent along with the data segments.



CHECKSUM

Receiver's End:

- All the four received segments along with received checksum, are added using 1's complement arithmetic to get the sum.
- The sum is again complemented.
- If the result comes as all 0s, the received data is accepted, otherwise the received data is discarded.



CHECKSUM PERFORMANCE

- The checksum **detects all errors involving an odd number of bits**.
- It also detects **most errors involving even number of bits**.
- Number of **overhead bits is also less** compared to two-dimensional parity check.

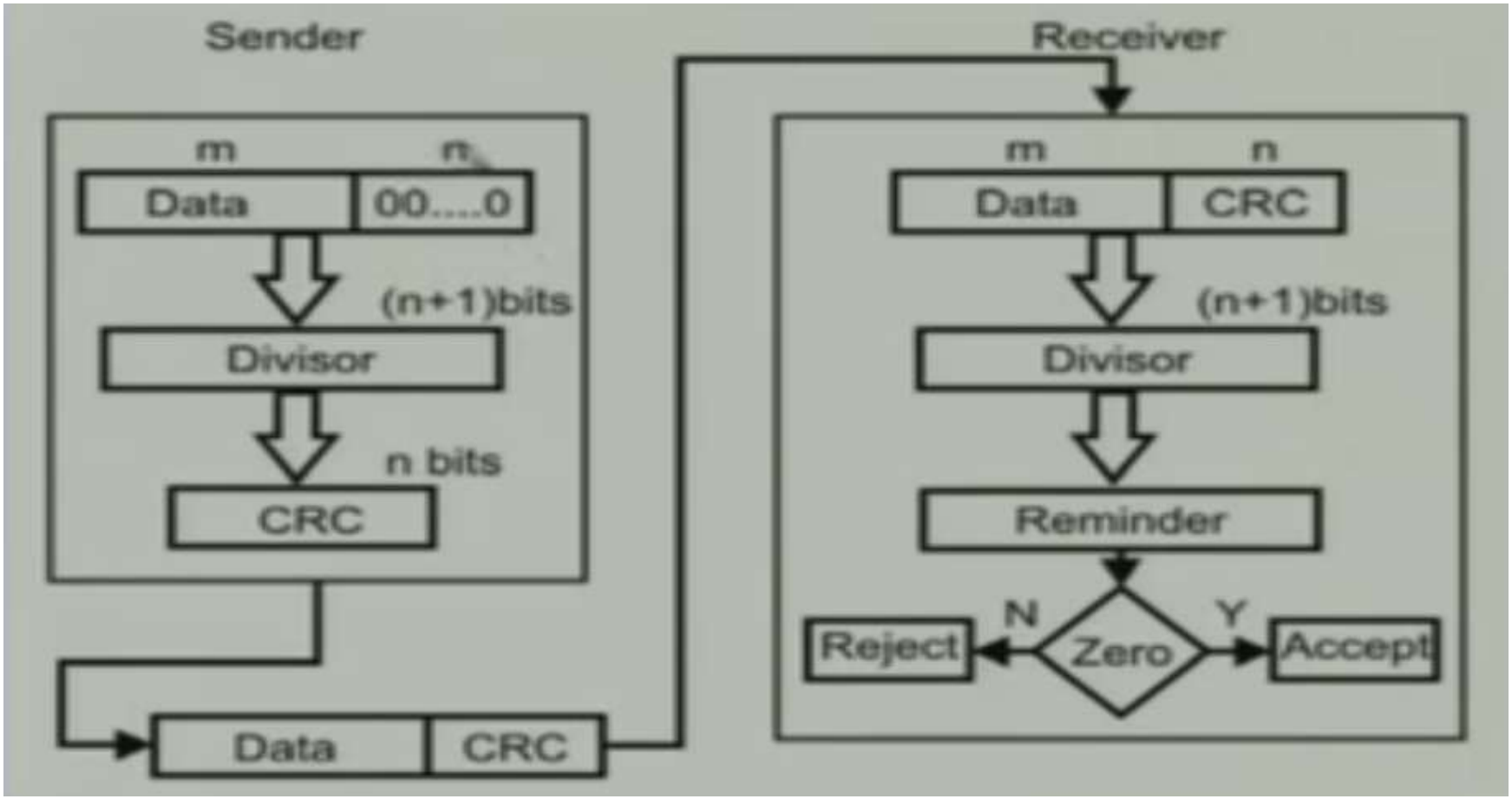
  e.g., In our example, 8 bits, the checksum were overhead.

  i.e., overhead bits is same in number as each block of data.

CYCLIC REDUNDANCY CHECK (CRC)

- CRC is one of the most powerful and commonly used error detecting codes.
- Given a **m-bits** - “block of bit sequence”, the sender generates a **n-bits** sequence, called as a **Frame Check Sequence (FCS),** so that the resulting frame, consisting of **m+n bits**, is exactly divisible by some **predetermined number**.
- The receiver divides the incoming frame by that number and if there is no remainder, it determines that there is no error.

CYCLIC REDUNDANCY CHECK (CRC)



XOR OPERATION WITH DIVISION AT SENDER

- Data (m) = 1010
- Divisor (**Predefined**) = 1011
- FCS (n)=000

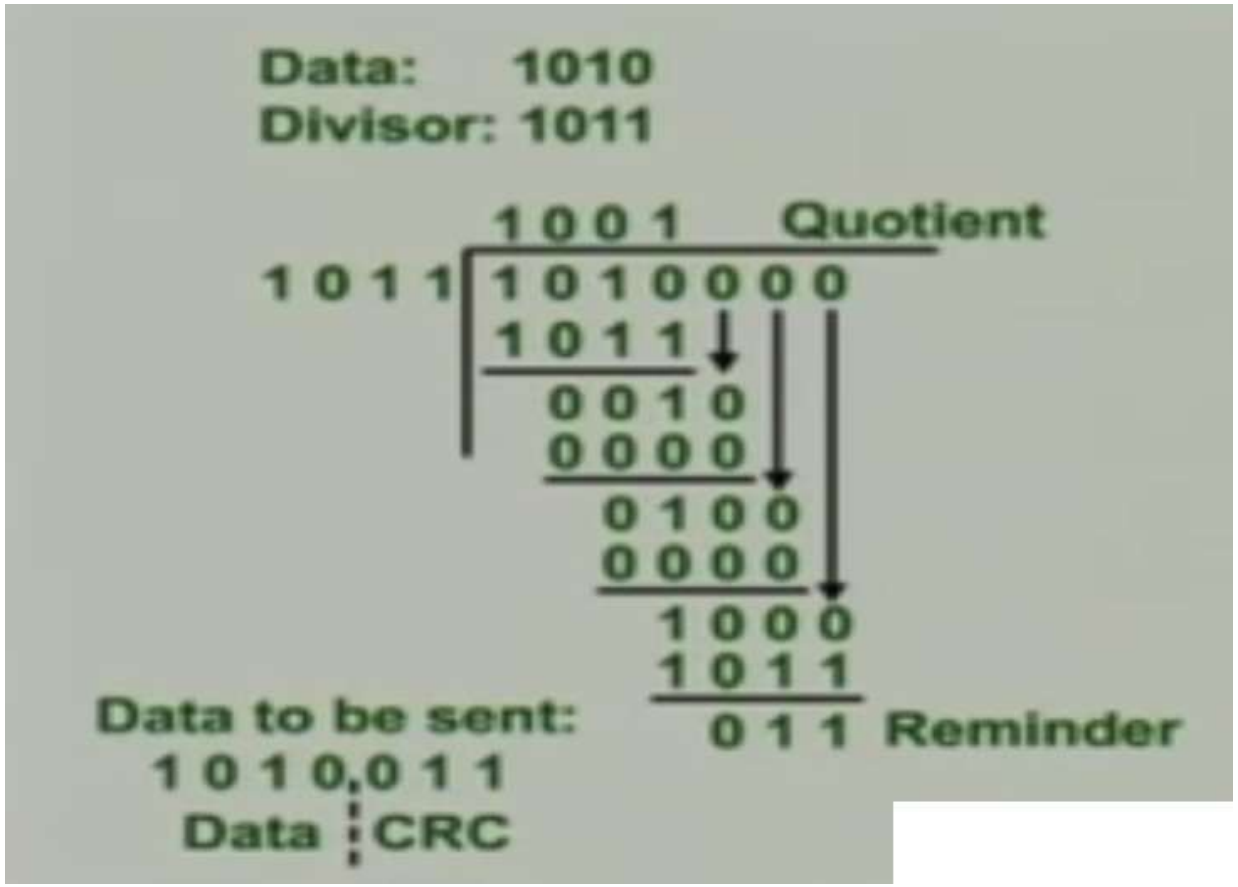
[1 bit less than the number of bits in divisor]

Thus, Dividend = 1010000

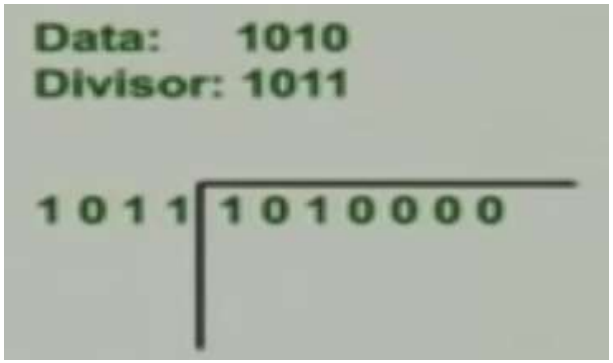
- *Data to send: 1010*
- *CRC to be sent :* ***011***

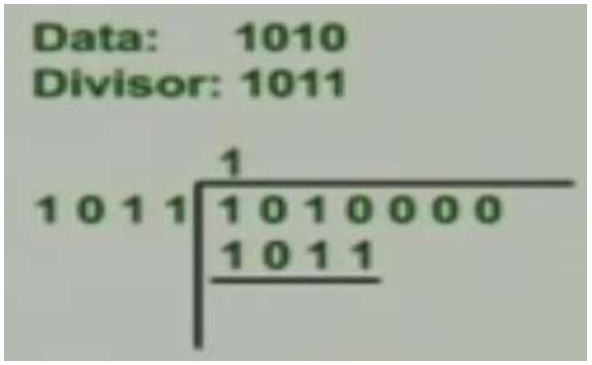
Data with CRC: 1010**011**

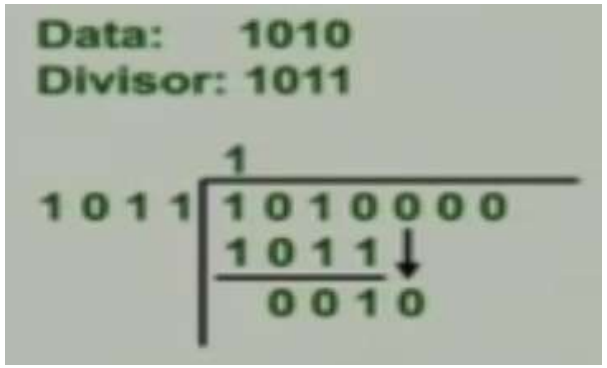
Receiver does similar operation for **1010011**(Dividend) and receives remainder as all 0's for error free transmission.

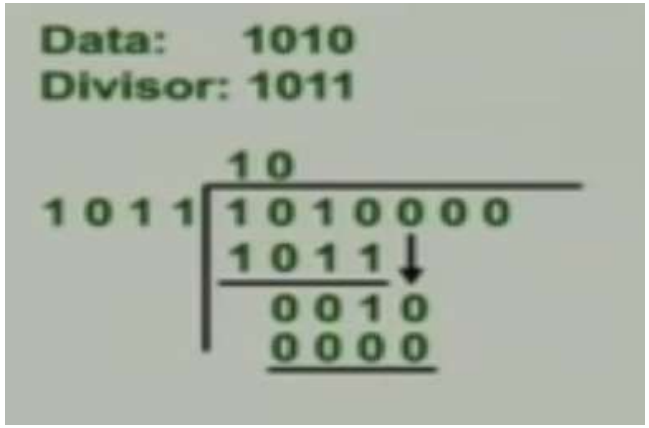


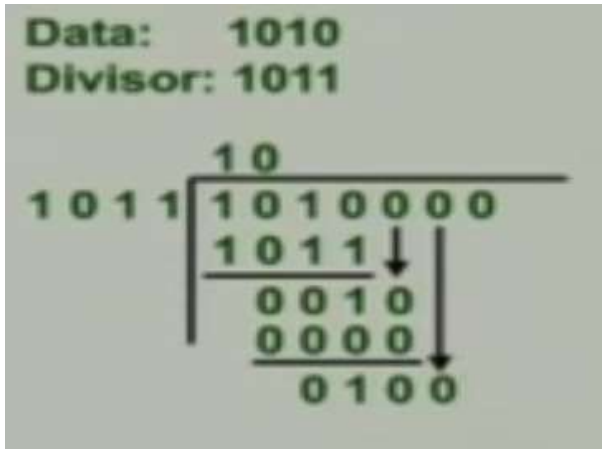
BITWISE DIVISION PROCESS WITH XOR

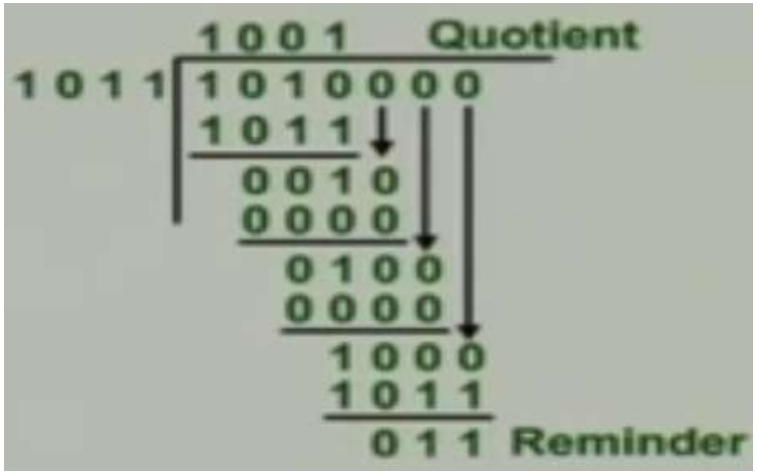












POLYNOMIAL REPRESENTATION

- All the values can be expressed as polynomials of a variable x.
  - P=11001 [**Divisor**] can be represented as:

    $P(X)=\mathbf{1*X^4 + 1*X^3 + 0*X^2 + 0*X^1+1 = X^4+X^3+1}$
- Thus, CRC process can be expressed as:
  - $[M(X)\ X^N] / P(X) = [Q(X) + R(X)] / P(X)$
  - Here, R(X) defines CRC bits.

M-Message is dividend
P-Polynomial is divisor
Q-Quotient
R-Remainder is CRC

- There are some standard polynomials based on some strong mathematical properties, which are used for the purpose of generating CRC bits.
- Commonly used divisor polynomials [P(X)] are:
  - CRC-16 = $\mathbf{X^{16}+X^{15}+X^2+1}$ **[**Commonly used in ATM**]**
  - CRC-CCITT = $\mathbf{X^{16}+X^{12}+X^5+1}$
  - CRC-32 = $\mathbf{X^{32}+X^{26}+X^{23}+ X^{22}+X^{16}+X^{12}+ X^{11}+X^{10}+X^8+ X^7+X^5+X^4+X^2+1}$

PERFORMANCE OF CRC

- CRC can detect:
  - All **single-bit errors** (similar to simple parity check).
  - All **double-bit errors** (if there are min. three 1's in characteristic polynomial).
  - **Any odd number of errors**.
  - **All burst errors** of less than degree of the polynomial .

    i.e., up-to 15-bit burst error can be detected if polynomial is of 16 bit
- CRC detects most of the larger burst errors with a high **probability but not all the burst errors**.
- For an example, "CRC-12" detects 99.97% of errors with a length of 12 bits or more.

ERROR CORRECTION

Two Approaches:

- Backward Error Correction
  - **Retransmission** is done if error is detected and acknowledged, i.e via ARQ (Automatic Repeat reQuest)

- Forward Error Correction
  - **Use of** more **redundancy in the transmitted data** helps not only in the detecting of error but correction, too.

  e.g. Hamming Code

HAMMING CODE (HC)

- Hamming Code is linear error correcting code.
- Hamming Code helps in one-bit and two-bit error detection while one-bit error correction **without detection of uncorrected errors -** which means it doesn't provide information about whether there are additional errors present in the block of data, beyond the one-bit error it corrected.
- HC can be applied to data units of any length.
- Here, all bit positions that are **Power of 2** are marked as a **Parity bits**.
- $2^0=1$, $2^1=2$, $2^2=4$, $2^3=8$, $2^4=16$, $2^5=32$ ……..and so on……....
- 1st , 2nd, 4th, 8th, 16th, 32nd , etc., positions are taken by **Parity bits.**
- Thus, Parity positions are: P_1 , P_2, P_4, P_8, P_{16}, P_{32} ……. and so on.....

HAMMING CODE

- Since, Parity positions are: P_1, P_2, P_4, P_8, P_{16}, P_{32} …….
- If the data to be sent is just four bits [D_x] = **1101** ***(an example)***
- Here, Positional values 2^0=**1**, 2^1=**2**, 2^2=**4,** etc. are taken by P_1, P_2 and P_4 and rest positions are taken by Data.
- Thus, Parity with Data sequence from MSB to LSB is D_7-D_6-D_5-P_4-D_3-P_2-P_1
- Substituting the values of given data from LSB:
  - D_7=**1**, D_6=**1**, D_5=**0**, P_4=?, D_3=**1,** P_2=?, P_1=? → **110**P_4**1**$P_2$$P_1$ **OR** $P_1$$P_2$ **1**P_4**011**
- The value of parity bits P_1 , P_2 , and P_4 is determined by the sequence of bits that is alternatively **checks** and **skips** in the sequence → P_1 P_2 $D_3$$P_4$$D_5$$D_6$$D_7$
  - For **position** P_1 → Check 1 bit; Skip 1 bit (i.e., 1, 3, 5, 7, 9, 11,…) → P_1 $D_3$$D_5$$D_7$
  - For P_2 → Check 2 bits; Skip 2 bits (i.e., 2, 3, 6, 7, 10, 11, 14, 15, …) → P_2 $D_3$$D_6$$D_7$
  - For P_4 → Check 4 bits; Skip 4 bits (i.e., 4, 5, 6, 7, 12, 13, 14, 15, …..) → $P_4$$D_5$$D_6$$D_7$

CHECK & SKIP with EVEN PARITY

Data & Parity Sequence [D7-D6-D5-P4-D3-P2-P1] with data values:

- D_7=1, D_6=1, D_5=0, P_4 =?, D_3=1, P_2 =?, P_1 =? ***[110$P_4$1P_2P_1]***

- Finding the Parity Bits (P1, P2 & P3) with the help of data bits & number sequence
- Finding value of P1: Check 1 bit, Skip 1 bit (i.e., 1, 3, 5, 7, 9, 11,…)
  - D_7=1, D_5=0, D_3=1, P_1 → 101P_1 → P_1= **0** **[Making the even parity]**
- Finding value of P2: Check 2 bits, Skip 2 bits (i.e., 2, 3, 6, 7, 10, 11, 14, 15, …)
  - D_7=1, D_6=1, D_3=1, P_2 → 111P_2 → P_2 = **1**
- Finding value of P4: Check 4 bits, Skip 4 bits (i.e., 4, 5, 6, 7, 12, 13, 14, 15, …..)
  - D_7=1, D_6=1, D_5=0, P_4 → 110P_4 → P_4 = **0**
- Thus, Hamming Code is:
  - D_7-D_6-D_5-P_4-D_3-P_2 -P_1 → ***110$P_4$1P_2P_1*** → **1100110**

Data → **1101** ***Data with Parity at the Sender*** → **1100110**

ERROR DETECTION IN HAMMING CODE

- Hamming Code Sent: **1100110**
- But Hamming Code Received: **1000110** **[D_7-D_6-D_5-P_4-D_3-P_2 -P_1]**
- For the value of P_1
  - Check 1 bit, Skip 1 bit (Positions →1, 3, 5, 7)
  - P_1-D_3-D_5-D_7 → 0101 → Even Parity : It's Fine, Put, P_1=**0**
- For the value of P_2
  - Check 2 bits, Skip 2 bits (Positions → 2, 3, 6, 7)
  - P_2-D_3-D_6-D_7 → 1101 → Odd Parity: Issue, For even parity, put, P_2=*1*
- For the value of P_4
  - Check 4 bits, Skip 4 bits (Positions → 4, 5, 6, 7)
  - P_4-D_5-D_6-D_7 → 0001 → Odd Parity: Issue, For even parity put, P_4=*1*

ERROR BIT CORRECTION IN "HC"

- After calculating parity, if we received:
- Even Parity – **Fine** – we put, ***P***x as 0 [In example, P_1= 0]
  - Odd Parity – Problem – we put ***P***x as 1 [In example, P_2=1 and P_4=1]
- As all values of Px were not equal to 0, we came to identify the error.
- Error Word: P_4-P_2-P1 → $P_4P_2P_1$ → 110
- Decimal Value is **6**
- Finally, error lies in the 6th bit position.
- Furthermore, we invert (flip) the 6th bit (starting from LSB to MSB)
- Data Received was → **1000110**
- After inverting 6th bit → **1100110**

Classwork:

Find Hamming Code for message (data)
"**01100111**"

Hamming Code for data “01100111”

- Data & Parity Sequence: D_{12}-D_{11}-D_{10}-D_9-P_8-D_7-D_6-D_5-P_4-D_3-P_2 -P_1 with data values:

D_{12}=0, D_{11}=1, D_{10}=1, D_9=0, P_8=?, D_7=0, D_6=1, D_5=1, P_4=?, D_3=1, P_2=?, P_1=? [**0110**P_8**011**P_4**1**P_2P_1]

- Finding the Parity Bits (P_8, P_4, P_2, **&** P_1) with the help of data bits:
- Finding value of P_1 : Check 1 bit, Skip 1 bit (i.e. Positions - 1, 3, 5, 7, 9, 11)
  - D_{11}=1, D_9= 0, D_7=0, D_5=1, D_3=1, P_1 → 10011P_1 → P_1 = **1**
- Finding value of P_2 : Check 2 bits, Skip 2 bits (i.e. Positions - 2, 3, 6, 7, 10, 11)
  - D_{11}=1, D_{10}=1, D_7=0, D_6=1, D_3=1, P_2 → 11011P_2 → P_2 = **0**
- Finding value of P_4 : Check 4 bits, Skip 4 bits (i.e. Positions - 4, 5, 6, 7, 12)
  - D_{12}=0, D_7=0, D_6=1, D_5=1, P_4 → 0011P_4 → P_4 = **0**
- Finding value of P_8 : Check 8 bits, Skip 8 bits (i.e. Positions - 8, 9, 10, 11,12)
  - D_{12}=0, D_{11}=1, D_{10}=1, D_9=0, P_8 → 0110P_8 → P_8 = **0**
- Thus, Hamming Code is:
  - D_{12}-D_{11}-D_{10}-D_9-P_8-D_7-D_6-D_5-P_4-D_3-P_2 -P_1 → **0110**P_8**011**P_4**1**P_2P_1 → **011000110101**
- ***Hamming Code =*** **011000110101 [Data with Parity Bits as per the sequence]**